HANDBUCH

GRAFIK-Interpreter: Prüfverfahren

Wie man prüft, dass der GRAFIK-Interpreter das Soll der Beschreibung erfüllt: Läufe, Tests, Abnahme

Dieses PDF gibt die Quelle im Wortlaut wieder; Listen, Tabellen und Codeblöcke entsprechen der Quelle, weich umbrochene Zeilen sind zu Absätzen zusammengeführt.

Quelle	C8205-Interpreter-GRAFIK_Pruefverfahren.md	0.95-028
Stand	10.11.2026	
Umfang	28 Seiten, A4 hoch	

> Maßgeblich bleibt die Quelldatei. Dieses PDF wird nicht automatisch nachgeführt: Es ist aktuell, solange die Version oben mit der Versionszeile der Quelldatei übereinstimmt.

C 8205 – GRAFIK-Interpreter: Prüfverfahren

Quelle: C8205-Interpreter-GRAFIK_Pruefverfahren.md · im Wortlaut der Quelle

Inhaltsverzeichnis

1 Zweck und Einordnung . . . 3

2 Prüfebenen . . . 4

3 Testskripte . . . 4

3.1 Sammellauf . . . 4

3.2 Die 20 Blöcke . . . 4

3.3 Skripte außerhalb des Sammellaufs . . . 7

4 Gleichwertigkeit TEST/GRAFIK . . . 7

5 Pult-Tastentest . . . 8

5.1 Zweck und Abgrenzung . . . 8

5.2 Das Verfahren in sechs Schritten (je Testfall) . . . 8

5.3 Voraussetzungen und Aufruf . . . 9

5.4 Die Testfälle (Pilotversuch PTK-001) . . . 9

5.5 Von der Tastenfolge zur Klickspur . . . 10

5.6 Auswertung . . . 11

5.7 Neuen Fall anlegen . . . 11

5.8 Grenzen . . . 12

5.9 Ergebnisse . . . 12

5.10 Verweise . . . 12

6 R300-Eingabe . . . 12

6.1 Bestandteile . . . 13

6.2 Aufrufe . . . 13

6.3 Was die Auswertung prüft (je Tipp der Klickspur) . . . 13

6.4 Erwartung und Ergebnisse . . . 13

6.5 Befund zu den Dateien der Klickspur und der Sollfolge . . . 14

7 Nur von Hand prüfbar . . . 14

7.1 Übersicht . . . 15

7.2 Handliste SW-Fenster (grafik_sw_tk_test.py --manuell --DEBUGLOG) . . . 17

7.3 Handliste Hörprüfung . . . 18

7.4 Trommel: Referenzwerte der Linux-Läufe (Dok. 02 §5) . . . 18

7.5 Handliste Lampenmittelung (H-12, OP-37) . . . 19

7.6 Trommelzeit der Befehlsschleife gegen die Dokumentation . . . 19

7.7 Zeitmessung: Pult-Fenster, Lampenabtastung und HW-Check gegen die Befehlszeit . . . 20

7.8 Einlesezeit des KEP und Trommelzeit . . . 21

8 Abnahmekriterien der Stufen 1 bis 5 . . . 21

8.1 Meilensteintabelle (S6-09, O6-6) . . . 22

8.2 Stufe 3 (Dok. 09 §4) . . . 22

8.3 Stufe 4 (Dok. 10 §5) . . . 23

8.4 Stufe 5 (Dok. 11 §4) . . . 23

8.5 Stufen 1 und 2 . . . 23

9 Windows (Teilabnahme 26.10.2026) . . . 24

9.1 Was bisher gilt (Quellen) . . . 24

9.2 Abnahmekriterien Windows (Vorschlag, aus den Quellen abgeleitet) . . . 24

9.3 Ergebnis der Teilabnahme (26.10.2026, Rechner des Projektinhabers) . . . 25

9.4 Noch offen unter Windows (nicht geprüft) . . . 26

10 Bekannte Befunde . . . 27

10.1 Probelauf vom 19.10.2026 (Entwicklungsumgebung) . . . 28

10.2 Kennungen . . . 28

C 8205 – GRAFIK-Interpreter: Prüfverfahren

Quelle: C8205-Interpreter-GRAFIK_Pruefverfahren.md · im Wortlaut der Quelle

Version: 0.95-028

Stand 19.10.2026 (Checkpoint G6-009): §1 bis §10 geschrieben, Entwurf, nicht abgenommen. Das PDF dazu (`C8205-GRAFIK_Pruefverfahren_A4.pdf`) wird mit `make_all_pdf.sh` erzeugt. Entstanden in Stufe 6 (Dokumentation konsolidieren); Plan und Anforderungen: Stufe-6-Beschreibung (Dokument 12, fällt nach Abschluss weg). Das Soll (was GRAFIK tun muss) steht in `C8205-Interpreter-GRAFIK_Beschreibung.md`; dieses Dokument sagt, **wie man prüft, dass er es tut**.

Leser: wer GRAFIK prüft oder abnimmt.

Alle Aufrufe gelten für den Projektordner als Arbeitsverzeichnis, unter Linux. Unter Windows ist davon nur ein Teil geprüft (→ §9).

1 Zweck und Einordnung

GRAFIK (`c8205_grafik.py`, Aufruf auch über `c8205_interp.py` ohne `--MODUS`) ist die Programmfamilie mit Pult-Fenster, SW-Fenster, Tönen und echter Trommel. TEST (`c8205_interp.py --MODUS=TEST`) ist der Stapelbetrieb ohne Fenster, ohne Töne und ohne C-Modul; alle Zeiten sind 0.

Regel der Gleichwertigkeit (F-04 a, entschieden 16.10.2026): Bei gleichem Programm liefern TEST und GRAFIK dieselben Ergebnisse: Registerinhalte, Flags, Ausgaben, Trommelinhalt. Ausgenommen sind die **Zeiten** und die **Befehlsentnahme nach X-Abbruch**. Wo ein Gerät in GRAFIK anders endet als in TEST, steht das als „gewollte Abweichung“ im Bericht (Leser am Bandende, → §4).

Rolle von TEST: TEST ist das Referenzverhalten. Weicht GRAFIK ab, wird der Fehler zuerst in GRAFIK gesucht (im Pult, im Rechner-Thread, in der Wiedergabe oder im Zeitverhalten), nicht in der Anforderung. Aus demselben Grund wird ein neuer Fall erst im TEST-Interpreter bestätigt (→ §5.7).

Gegenstand	Wo beschrieben
Testformat, Tastenfolgen und Prüfungen des TEST-Interpreters	`C8205-Interpreter-Durchführung_eines_Tests.md` (dort steht ein Verweis auf dieses Dokument)
Soll-Verhalten von GRAFIK	`C8205-Interpreter-GRAFIK_Beschreibung.md`
Prüfen von GRAFIK, Abnahme, Handlisten	dieses Dokument

Prüfweg (F-04 b): Der Projektinhaber hatte auf die Frage nach dem Prüfweg („nur von Hand“ oder „Einspielmodus“) mit „beide müssen die gleichen Ergebnisse liefern“ geantwortet; vorläufig galt Handabnahme. Seither gibt es beides: skriptbare Läufe (Klickspur, `--WIEDERGABE`, Debug-Log) und die Handlisten in §7. Was nur von Hand geht, steht in §7.

Ein Kern: Zeitkritische Prüfungen nicht gleichzeitig starten (`grafik_alle_tests.sh` sagt das im Kopf); Zeitgrenzen reißen sonst.

2 Prüfebenen

Ebene	Was sie prüft	Was sie nicht prüft	Wer kann sie laufen lassen
1 Logiktests ohne Fenster (§3, Blöcke 1 bis 12, 14 bis 16 und 19)	Kern, Geräte, Pult-Steuerung, SW-Logik, Töne (Mischer, Klangwahl, Zeiten) gegen Ersatzmodule für Tk und Audio	Aussehen, echtes Tk, Hören	Entwickler und Projektinhaber; kein Bildschirm nötig
2 Gleichwertigkeit TEST/GRAFIK (§4)	dieselben Ergebnisse in beiden Modi, je Test aus `Tests/`	Fenster, Töne, Zeiten	beide; dauert am längsten
3 Trockenläufe (Block 13, `--ohne-tk`, `r300_eingabe_trockenlauf.py`)	Skripte, Klickspuren, Pult-Modell, Rechner-Thread im selben Prozess **ohne Tkinter**	Tkinter selbst	beide
4 Tk-Tests am echten Fenster (§5, §6, `grafik_sw_tk_test.py`)	Klick → Pult-Steuerung → Rechner-Thread → Zustand im laufenden Fenster; Tasten im SW-Fenster; HW-Check	Aussehen, Gehör, Treffer- und Fokusprüfung des Pult-Fensters (die Wiedergabe umgeht das Tk-Mausereignis)	nur wer Tkinter und Bildschirm hat (bisher der Projektinhaber)
5 Handläufe und Hörprüfung (§7)	Aussehen, Klang, Taktgefühl, Fokus, Windows	–	Projektinhaber, Tester

Umgebung des Entwicklers: kein Tkinter, kein Audiogerät. Ebene 4 und 5 sind dort nicht möglich; in den Berichten steht dann „Trockenlauf“, nicht „echter Lauf“. Wer ein Ergebnis weitergibt, nennt immer die Ebene.

Wiedergabe-Artefakte: Die Wiedergabe erzeugt Tastenereignisse mit `event_generate`. Das ist nicht ein Mensch an der Tastatur. Bekannt: Der Gravis (Tottaste) lässt sich nicht erzeugen (F-56); die Haltezeit der Taste und der AltGr-Zustand sind nachgebildet (120 ms, `state=0x80`, nur unter Linux). Ein Fehler der Ebene 4 ist deshalb zuerst daraufhin zu prüfen, ob er von der Wiedergabe kommt (Auswertung `r300_eingabe_log_auswerten.py` unterscheidet das ausdrücklich, Punkte 1 und 2).

3 Testskripte

3.1 Sammellauf

```
bash testscripte/grafik_alle_tests.sh            alle 20 Blöcke
bash testscripte/grafik_alle_tests.sh --schnell  ohne die beiden Gleichwertigkeitsläufe (18 Blöcke)
```

Je Block eine Zeile `OK` oder `FEHLER` mit Laufzeit, bei Fehler die letzten 40 Ausgabezeilen; am Ende „Blöcke bestanden: n, fehlgeschlagen: m“. Rückgabewert 0 nur, wenn alle Blöcke bestanden sind. Das Skript setzt für sich `C8205_FENSTERLAGE=aus` (die Tests dürfen die gemerkte Fensterlage des Benutzers nicht beschreiben), `PYTHONDONTWRITEBYTECODE=1` und `PYTHONWARNINGS=ignore::ResourceWarning` (die Warnungen stammen aus dem unveränderten `c8205_interp.py`).

3.2 Die 20 Blöcke

Die Spalte „Erwartung“ ist bei allen: Rückgabewert 0 (alle Prüfungen des Skripts bestanden). Zahlen in Klammern sind die Prüfungszahlen zum Zeitpunkt der Abnahme laut Quelle, nicht neu gezählt.

Nr.	Block	Skript (Aufruf `python3 testscripte/…`)	Zweck
1	Kern	`grafik_kern_tests.py`	Dauerbetrieb am Pult, Unterbrechung durch H, Eingabe über die Warteschlange mit zwei Quellen (T/SW) aus einem zweiten Thread, Abbruch, Bahnsperre, Schalterdatei über `GrafikTrommel`, echte Wartezeit

C 8205 – GRAFIK-Interpreter: Prüfverfahren

Quelle: C8205-Interpreter-GRAFIK_Pruefverfahren.md · im Wortlaut der Quelle

Nr.	Block	Skript (Aufruf `python3 testscripte/…`)	Zweck
2	Geräte	`grafik_geraete_tests.py`	Leser (Ordner, Änderungszeit, `LOCHSTREIFENKORB`, aktives Warten), Stanzer (`stanzer.ls` fortschreiben), Zusammenspiel mit dem Rechner
3	Checkpoint 4	`grafik_checkpoint4_tests.py`	Gerätefehler brechen den Rechner nicht ab (Lauf endet mit Meldung, danach H G R neu), H bei Endlosschleife mit Ausgabebefehlen, Reaktionszeit von H bei echter Wartezeit, Stanzer legt an und schreibt fort
4	Checkpoint 5	`grafik_checkpoint5_tests.py`	`KonfigFehler` beim Verschieben in den `LOCHSTREIFENKORB` (letztes Lochbild wird nicht geliefert, Datei bleibt liegen)
5	Stufe 2	`grafik_pult_tests.py` (mit `-I`)	Pult-Steuerung ohne Tkinter, Tk-Schicht gegen Ersatzmodul
6	Stufe 3 SW-Fenster	`grafik_sw_tests.py` (mit `-I`)	SW-Logik, Anbindung an die K0-Warteschlange, Tk-Schicht gegen Ersatzmodul (90 Prüfungen, Stand 21.10.2026; vorher 81; neu: zwei Tests zum ersten Zeichen nach dem Einschalten, OP-09; davor drei zur Fokus-Entprellung, OP-32)
7	Stufe 3 Ende-zu-Ende	`grafik_sw_ende_zu_ende.py` (mit `-I`)	`ziffern` und `sw_zwei_ziffern` auf dem echten GRAFIK-Rechner, SW-Fenster durch die Logikschicht ersetzt; Ausgabe, Endzustand, Zeichenlage GB (15 Prüfungen)
8	Stufe 3 Pult-Tasten	`grafik_sw_pult_test.py` (mit `-I`)	SW-Eingabe über Netzschalter, Hochlauf, Taste SW, G, R; Lage des SW (5 Prüfungen)
9	Stufe 4 Töne	`grafik_stufe4_tests.py` (mit `-I`)	Mischer, Schreibwerkzeit, Klangwahl, Selektor SW, angezeigte Eingabe, Ausfall (Code 16); ohne Audiogerät, Zeitgeber und Zufall ersetzt (90 Prüfungen, Stand 21.10.2026; vorher 80, die Angabe 69 war veraltet; neu fünf Tests zum Klang von WR, OP-42; mit `C8205_MISCHER=C` auch gegen das C-Modul)
10	Stufe 5 `--MODUS`	`grafik_stufe5_tests.py` (mit `-I`)	Standard GRAFIK, TEST wie bisher, Wrapper; Unterprozesse mit eigener Trommelkopie; GRAFIK mit Ersatzmodul und leerem PATH, Start endet mit Code 16
11	Tastaturbelegung	`grafik_tastatur_tests.py` (mit `-I`)	`--TASTATUR de|en` (G4-011)
12	Abstand Umschalt-/Zeichentaste	`grafik_sw_abstand_tests.py` (mit `-I`)	Abstand bei gehaltener Umschalttaste (G4-011)
13	Trockenlauf Tk-Test	`grafik_sw_tk_test.py --ohne-tk --bericht /tmp/sw_tk_trocken.txt`	Trockenlauf des Tk-Tests ohne Tkinter (der echte Lauf steht in §3.3)

C 8205 – GRAFIK-Interpreter: Prüfverfahren

Quelle: C8205-Interpreter-GRAFIK_Pruefverfahren.md · im Wortlaut der Quelle

Nr.	Block	Skript (Aufruf `python3 testscripte/…`)	Zweck
14	Stresstest Pult/Rechner	`grafik_pult_stress.py 6 <Seed>` für Seed 3, 5, 6	Zufallsklicks gegen den echten Rechner mit Rechner-Thread; hängt ein Klick länger als 3 s, werden alle Threads ausgegeben (Exit 1)
15	Trommel-Erweiterungsmodul	`cd c-module/python && python3 -m unittest test_c8205_trommel`	Python-Seite des C-Moduls
16	C-Module	`cd c-module && make test-alle && make clean`	drei Testrahmen (Trommel, Koinzidenz, `tonmisch`); `make clean` räumt die Objekte auf; die vorübersetzte `c8205_trommel.abi3.so` blieb im Probelauf (§10.1) byte-gleich
17	Gleichwertigkeit `--alle`	`grafik_gleichwertigkeit.py --alle`	→ §4
18	Gleichwertigkeit neue Geräte	`grafik_gleichwertigkeit.py --neue-geraete`	→ §4
19	Trommelwächter, Selbsttest	`grafik_trommel_waechter_tests.py` (mit `-I`)	elf Prüfungen des Wächters mit Gegenproben in einem Temp-Ordner: unverändert → 0; geänderte, gelöschte oder neu entstandene `trommel.txt` bzw. `trommel.basp` → 1; Stand fehlt oder Aufruf falsch → 2 (OP-19, Variante b, 21.10.2026)
20	Trommelwächter, Vergleich	`grafik_trommel_waechter.py pruefen <Stand>`	Letzter Block des Sammellaufs: vergleicht die Prüfsummen von `Trommel/trommel.txt` und `Trommel/trommel.basp` mit dem Stand, den das Skript vor dem ersten Block gemerkt hat; **FEHLER, wenn sich eine der beiden Dateien während des Laufs geändert hat** (die Datei wird nicht zurückgesetzt). Gilt auch bei `--schnell`

Die Nummern 1 bis 20 zählen die Blöcke in der Reihenfolge des Skripts (bei `--schnell` fehlen 17 und 18, 19 und 20 laufen trotzdem); Block 13 ist im Skript der Trockenlauf des Tk-Tests, Block 14 der Stresstest.

3.3 Skripte außerhalb des Sammellaufs

Skript	Zweck und Aufruf	Erwartung	Ebene
`grafik_pult_tk_test.py`	Pult-Tastentest am echten Fenster, → §5	Rückgabewert 0, `bericht.txt` ohne FEHLER	4 (`--ohne-tk`: 3)
`grafik_sw_tk_test.py`	Test des echten SW-Fensters **und** des HW-Check-Fensters in einem Prozess; Version SWTK-004. Aufruf `python3 testscripte/grafik_sw_tk_test.py [--ohne-tk] [--manuell] [--DEBUGLOG [DATEI]] [--bericht DATEI]`. Mit `--manuell` folgt nach dem automatischen Teil die Handliste (→ §7.2). Die Tastenzustände für Großbuchstaben, `§` (Umschalt) und `@ € [ ] { } \ | ~` (AltGr) setzt `zustand_x11()` **nur unter Linux**	Ergebnis je Zeile `OK`, `WARNUNG`, `FEHLER`; Abnahmestand: 49 OK, 1 WARNUNG (HW-Check-Fokus, B-5), 0 FEHLER	4
`r300_eingabe_*`	Eingabeprogramm „alle Zeichen des R300-Codes“, → §6	siehe §6	3, 4
`grafik_sw_vormerkung_s3_13.py`	Prüfmittel VM-003 zu S3-13/S3-18 (keine Vormerkung, Sperre statt Warten; Eingabe nur bei aktivem `011`). Ändert keinen Programmcode. **Nicht** Teil des Sammellaufs	Rückgabewert 0 (4 von 4, ohne `expectedFailure`)	1
`k0_schatten_haken.py`	Messwerkzeug zu 0.03-016: baut in eine **Kopie** von `c8205_interp.py` einen Zusatz, der Anwahlbefehle K0 mit abweichender Entscheidung von alter und neuer Regel in die Datei aus `C8205_SCHATTEN` schreibt. **Nur auf eine Projektkopie anwenden**	kein Prüfskript, kein Rückgabewert	–
`erzeuge_leere_trommel.py`	erzeugt eine leere Trommeldatei	–	–
`c8205_bis_test.sh`, `compute_all_tests.sh`	Stapelläufe der TEST-Seite (BIS-Bausteintests, Regressionsschleife; die früheren `test.sh`, `test_bis.sh`, `test2_bis.sh`, `test3_bis.sh` sind entfallen); beschrieben in `Durchführung_eines_Tests` und der Projektstruktur, hier nicht wiederholt	–	TEST

Reihenfolge nach einer Änderung: zuerst `grafik_alle_tests.sh --schnell`, danach der volle Lauf. Der volle Lauf mit den Gleichwertigkeitsläufen braucht am Rechner des Projektinhabers einige Minuten (Block 17 allein 159 s, Stand 19.10.2026).

4 Gleichwertigkeit TEST/GRAFIK

`testscripte/grafik_gleichwertigkeit.py` ist das Abnahmewerkzeug für Stufe 1. Es spielt vorhandene Tests aus `Tests/` zweimal ab und vergleicht:

Lauf	Verfahren
(A)	bisheriges TEST-Verfahren: `c8205_interp.lauf`, rechnerische Trommel, alle Zeiten 0
(B)	GRAFIK-Kern: `c8205_grafik.Rechner`, echte Trommel über das C-Modul, Tasten einzeln, K0-Zeichen über die Warteschlange
(C)	wie (B), aber Leser und Stanzer sind die neuen Klassen `GrafikLeser`/`GrafikStanzer` (nur Tests, die Leser oder Stanzer benutzen)

Verglichen werden Register und Flags, Trommelinhalt (Datei, Byte für Byte), Schreibwerk-Ausgabe, Stanzerdatei und das Ergebnis der Prüfungen der Testdatei. **Ausgenommen sind die Zeiten.** Jeder Lauf geschieht in einem eigenen Arbeitsordner mit eigener Kopie der Trommeldatei; die gemeinsame `trommel.txt` bleibt unberührt.

```
python3 testscripte/grafik_gleichwertigkeit.py --alle               alle Tests aus Tests/ (A gegen B)
python3 testscripte/grafik_gleichwertigkeit.py TEST ...             nur genannte Tests
python3 testscripte/grafik_gleichwertigkeit.py --neue-geraete       (C): nur Tests mit Leser oder Stanzer
        --zeit SEK    Zeitgrenze;   --bericht DATEI    Bericht in eine Datei
```

Erwartung: keine Abweichung. Die einzige gewollte Ausnahme ist am Bandende des Lesers: GRAFIK wartet dort aktiv (Review F-11), TEST endet mit Meldung. Das Werkzeug bricht das Warten nach 1,5 s ab und meldet es als

`ABWEICHUNG GEWOLLT`, und zwar nur dort, wo TEST am Bandende mit Meldung endet. Jede andere Abweichung ist ein Befund.

Stand der Abnahme: `--alle` und `--neue-geraete` bestanden im Gesamtlauf vom 19.10.2026 (Projektinhaber, 18 von 18 Blöcken, A5-1, A5-5).

Schreibwerkzeit in den Tests (OP-40): Der Sammellauf und die Gleichwertigkeitsläufe benutzen das Schreibwerk nicht und warten daher keine Schreibwerkzeit ab. Wie viel Zeit sie bei einem Lauf über den echten Treiber kostete, rechnet `python3 testscripte/grafik_sw_zeit_messung.py` aus der Schreibwerk-Ausgabe der Tests aus (Stand 21.10.2026: 33 von 199 Tests, 1233 Zeichen, rund 110 s). Das Skript gehört nicht zum Sammellauf.

5 Pult-Tastentest

(Eingegliedert aus `C8205-Interpreter-Pult_Tastentest_GRAFIK.md`*, Version 0.03-001. Die Verweise sind umgestellt (19.10.2026); die alte Datei entfällt mit S6-11.)*

Das Verfahren prüft die **Tastenbedienung des Bedienpultes** im GRAFIK-Interpreter mit dem **echten Tkinter-Fenster**. Es benutzt das vorhandene Testformat des TEST-Interpreters für die Anforderung und legt eine Klickspur darüber. Skript: `testscripte/grafik_pult_tk_test.py`, Version PTK-001. Stand: Pilotversuch am 18.10.2026 mit allen 8 Fällen im echten Fenster bestanden (→ §5.9).

5.1 Zweck und Abgrenzung

Vor diesem Verfahren gab es keine automatische Prüfung, die das Pult-Fenster mit echtem Tkinter bedient:

Prüfung	Was sie abdeckt	Tkinter
`grafik_pult_tests.py` (Block 5)	Pult-Steuerung und Tk-Schicht gegen ein **Ersatzmodul**: Verdrahtung und Aufrufe, nicht Aussehen und Ablauf	nein
`grafik_pult_stress.py` (Block 14)	zufällige Klicks gegen den echten Rechner mit Rechner-Thread: bleibt etwas hängen?	nein
`grafik_gleichwertigkeit.py` (§4)	Gleichwertigkeit TEST/GRAFIK, Tasten einzeln über den GRAFIK-Kern	nein
`grafik_pult_tk_test.py`	Tastenfolgen am **echten Pult-Fenster**: Netzablauf, Klicks, Rechner-Thread, Takt, Beenden; Wirkung auf Rechnerzustand und Speicher	**ja**

Geprüft wird der Weg **Klick → Pult-Steuerung → Rechner-Thread → Rechnerzustand** im laufenden Fenster, gegen ein **Soll, das der TEST-Interpreter bestätigt hat**. Die Handabnahme des Aussehens ersetzt es nicht (§5.8).

5.2 Das Verfahren in sechs Schritten (je Testfall)

1. **TEST-Anforderung.** Das Skript schreibt eine Testdatei im Format von `Durchführung_eines_Tests` (Abschnitte 2, 3, 3a): `#TROMMEL=LEER`, die Zellen der Anfangsbelegung, die Tastenfolge, die `Prüfungen:` (Zustandszeile und Speicherzellen).
2. **TEST-Lauf.** `c8205_interp.py` führt die Testdatei auf einer eigenen Trommelkopie aus. **Nur bei Rückgabewert 0** gilt die Trommel als geeignet initialisiert und das Soll als richtig; sonst wird der Fall im GRAFIK-Interpreter nicht versucht.
3. **Anfangs-Trommel und Klickspur.** Das Skript schreibt die Anfangs-Trommel (alle 4096 Zellen, nicht genannte = 0) und eine Klickspur (Format `C8205-Klickspur 1`, §5.5): Netzschalter ein, die Tastenfolge als Pult-Klicks mit Zeitabständen, Netzschalter aus.
4. **GRAFIK-Lauf.** Aufruf (vom Skript gebildet): `python3 c8205_grafik.py --STUMM --netz-schnell --BASP 000000 --trommel <Kopie> --leser1 … --leser2 … --stanzer1 … --WIEDERGABE <Klickspur> --DEBUGLOG <debug.log>`. Das Pult-Fenster öffnet sich, spielt die Klickspur selbst ab und **endet mit dem Netzschalter-Aus von selbst**. Zeitgrenze: Dauer der Klickspur + 40 s; danach wird der Prozess beendet und der Fall gilt als Fehler.
5. **Auswertung** nach dem Ende des Interpreters aus Debug-Log und Trommeldatei (§5.6).
6. **Ergebnis:** `bericht.txt` und ein ZIP des Ergebnisordners zum Zurückschicken.

5.3 Voraussetzungen und Aufruf

Voraussetzungen: Python 3 mit Tkinter und ein Bildschirm (oder `--xvfb`), das gebaute C-Modul `c8205_trommel` in `c-module/python/` (wie für `c8205_grafik.sh`), die Ordner `pult/` und `markdown-dateien/`.

```
python3 testscripte/grafik_pult_tk_test.py                    alle Fälle (rund eine Minute)
python3 testscripte/grafik_pult_tk_test.py --faelle pk-001,pk-004
python3 testscripte/grafik_pult_tk_test.py --xvfb              Linux ohne Bildschirm (xvfb-run)
python3 testscripte/grafik_pult_tk_test.py --langsam 2         alle Zeitabstände verdoppeln
python3 testscripte/grafik_pult_tk_test.py --ohne-tk           Trockenlauf im selben Prozess ohne Fenster
python3 testscripte/grafik_pult_tk_test.py --nur-test          nur Schritt 1 und 2
python3 testscripte/grafik_pult_tk_test.py --ausgabe ORDNER    Ergebnisordner
```

Schalter	Wirkung
`--faelle`	kommagetrennte Namensanfänge, z. B. `pk-001,pk-004`; Standard: alle
`--langsam F`	Faktor für alle Zeitabstände der Klickspur (Standard 1); für langsame Rechner
`--xvfb`	startet GRAFIK über `xvfb-run -a` (virtuelles Display)
`--ohne-tk`	**Trockenlauf:** dieselbe Wiedergabe im selben Prozess ohne Fenster; prüft Skript, Klickspur, Pult-Modell und Rechner-Thread, **nicht Tkinter**
`--nur-test`	nur TEST-Anforderung schreiben und mit dem TEST-Interpreter prüfen

Während des Laufs das Fenster nicht anfassen. Die Wiedergabe ignoriert Maus und Tastatur (Log: `MAUS … ignoriert: Wiedergabe laeuft`). Strg+C beendet den laufenden Interpreter; Bericht, Summe und ZIP werden trotzdem geschrieben (`ABBRUCH`). Rückgabewert: 0 nur, wenn alle gewählten Fälle bestanden haben, sonst 1 (2 bei unbekanntem Fall).

Ergebnisordner (Standard `Test-Protokolle/pult_tk/<Zeitstempel>/`, daneben `<Zeitstempel>.zip`):

Datei	Inhalt
`bericht.txt`	alle Prüfungen je Fall, OK/FEHLER, Summe; wird nach jedem Fall fortgeschrieben
`<Fall>/<Fall>.test`	die erzeugte TEST-Anforderung
`<Fall>/test_ausgabe.txt`, `test_protokoll/`	Ausgabe und Protokoll des TEST-Laufs
`<Fall>/trommel_anfang.txt`, `trommel.txt`	Anfangs-Trommel; die Trommel **nach** dem GRAFIK-Lauf
`<Fall>/klickspur.txt`	die abgespielte Klickspur
`<Fall>/debug.log`	Debug-Log des GRAFIK-Laufs (jede Zeile sofort geschrieben, auch bei Absturz lesbar)
`<Fall>/grafik_stdout.txt`, `grafik_stderr.txt`	Konsolenausgabe des Interpreters

Die gemeinsame Trommel `Trommel/trommel.txt` und die Geräteordner des Projekts bleiben unberührt: Jeder Fall läuft auf einer eigenen Kopie mit eigenen, leeren Ordnern `Leser1`, `Leser2`, `Stanzer1`.

5.4 Die Testfälle (Pilotversuch PTK-001)

Die Fälle stehen im Skript in der Liste `FAELLE`. `pk-001` bis `pk-008` sind aus den TEST-Tests `Tests/tk-001` bis `tk-008` abgeleitet.

Fall	Tastenfolge	Was geprüft wird	Soll (Endzustand)
`pk-001_H_G_M_R`	H G M R	Grundstellung, Sprung mit Rückkehrabsicht (BR = 0000 025), Lauf bis Stopp	BZ 0001, AR 0005, AC 0, US 1
`pk-002_H_G_Mplus_E_Mminus_M_R`	H G M+ E M- M R	E unter M (Halter), danach M, Lauf	BZ 0002, AC 00000004000, AR 0010
`pk-003_H_G_UE_R`	H G Ü R	Ü stellt 7777 240 ein, R transportiert AC nach 7777	BZ 0001, AR 0005; Zelle 7777 = 0
`pk-004_H_G_Mplus_E_Mminus_UE_R`	H G M+ E M- Ü R	E unter M, Ü, R mit Transport	BZ 0002, AC 00000002000, AR 0010; Zelle 7777 = AC

Fall	Tastenfolge	Was geprüft wird	Soll (Endzustand)
pk-005_H_G_Mplus_E_Mminus_ST_R	H G M+ E M- ST R	ST nimmt die Stopp-Wirkung zurück, Lauf bis zum zweiten Stopp	BZ 0003, AC 00000002000, AR 0020
pk-006_ST_R_ohne_Stopp	ST R	ST ohne vorherigen Stopp ist wirkungslos	BZ 0001, AR 0005
pk-007_R_UE_Rollensperre	R Ü	Stellung R sperrt Ü (Rollensperre); Zelle 7777 bleibt unverändert	wie pk-006; Zelle 7777 unverändert
pk-008_H_G_UE_E	H G Ü E	Einzelschritt E, hält vor Knoten 19	BZ 0001, AR 0005; Zelle 7777 = 0

Gemeinsame Zustandszeile der Prüfungen (außer wo oben anders): FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0.

Abweichung bei pk-007: tk-007 („Ü ohne H wirkungslos“) lässt sich im Pult nicht nachstellen, weil **H beim Einschalten schon steht**. Der Fall prüft stattdessen die **Rollensperre** (Pult-Eigenschaft). Der TEST-Interpreter kann „R Ü“ nicht ausdrücken (Rückgabewert 16, Tastenfolge nicht gültig); der Fall läuft deshalb **ohne TEST-Absicherung**, das Soll ist von Hand gesetzt, und der Bericht vermerkt das (HINWEIS TEST-Interpreter entfaellt …). Solche Fälle tragen im Skript den Schlüssel ohne_test.

5.5 Von der Tastenfolge zur Klickspur

Ausgangszustand im Pult. Beim Einschalten des Netzes (Hochlauf, dann B grün) setzt der Interpreter H und SW=0 und startet den Rechner-Thread. Eingerastet sind H, T und K0. Ein Klick auf H ist deshalb am Anfang **wirkungslos** (wirkt=0), der Rechner bekommt dafür kein Token.

Umsetzung der Tasten:

Taste der Tastenfolge	Klicks in der Klickspur	Tokens an den Rechner (Debug-Log TOKEN)
H	Klick H	H, wenn nicht schon H steht (sonst wirkungslos)
R	Klick R, danach lange Pause (Lauf bis Halt)	R; wirkungslos bei gehaltenem M/E oder wenn R schon steht
M, E	Klick, 0,15 s später Loslassen	M+, M- bzw. E+, E-
M+, M-	Umschalt+Klick (Halter ein bzw. aus)	M+ bzw. M-
G, Ü (UE), ST, K0 bis K3	Klick	gleichnamig (UE für Ü)

In Stellung R sind E, M, G, Ü (und T, SW) gesperrt: Der Klick bleibt ohne Wirkung und ohne Token (Rollensperre). Andere Tasten (SP, BS, LS, SW, T, Eingabetasten) setzt das Skript noch **nicht** in Klicks um; es bricht mit einer klaren Meldung ab (§5.8).

Zeitabstände (Sekunden, mit --langsam multipliziert; Konstanten am Anfang des Skripts; mit --STUMM --netz-schnell dauern Hochlauf und Auslauf je 0,3 s):

Konstante	Wert	Bedeutung
T_START	1,0	erster Tastenklick nach dem Netzschalter
T_TASTE	0,6	nach einer Taste
T_E	1,0	nach E (Einzelschritt im Rechner-Thread)
T_R	2,5	nach R (Lauf bis zum Halt)
T_LOS	0,15	Klick auf M oder E bis zum Loslassen
T_NETZ_AUS	0,8	letzte Taste bis Netzschalter aus

Gemessen: Der Rechner braucht nach einer Taste einige Millisekunden, die Reserve ist groß. Bei **längeren Programmen nach R** muss T_R (oder --langsam) erhöht werden.

Format der Klickspur (Textdatei `C8205-Klickspur 1`; Beschreibung im Kopf von `grafik_debuglog.py`; von Hand änderbar):

```
# C8205-Klickspur 1
# fall: pk-004_H_G_Mplus_E_Mminus_UE_R
# trommel_md5: <MD5 der Anfangs-Trommel>
    0.000 klick NETZ wirkt=1
    1.000 klick H wirkt=0
    1.600 klick G wirkt=1
    2.200 klick M umschalt wirkt=1
    2.800 klick E wirkt=1
    2.950 los E
    ...
    7.700 klick NETZ wirkt=1
```

Das Skript trägt für **jeden Klick die erwartete Wirkung** (`wirkt=`) aus einem kleinen Modell (Stellung H/R, Halter, Rollensperre) ein. Die Wiedergabe vergleicht sie mit der tatsächlichen Wirkung und meldet jede Abweichung im Log (`ABWEICHUNG`). Sie ruft die Pult-Steuerung auf wie ein Mausklick, aber **ohne Tk-Mausereignis**: Fokus und Trefferprüfung des Fensters sind nicht dabei.

5.6 Auswertung

Nach dem Ende des Interpreters prüft das Skript je Fall aus `debug.log`, `trommel.txt` und `grafik_stderr.txt`:

Prüfung	Bedingung für OK
Ende	Interpreter beendet sich selbst, Rückgabewert 0 (nicht Zeitgrenze)
Log	beginnt mit `START`
Wiedergabe	beginnt mit der erwarteten Zahl Ereignisse und endet mit `0 Abweichung(en) der Wirkung`; keine Zeile mit `ABWEICHUNG`
Tokens	die Folge der `TOKEN`-Zeilen ist genau die erwartete (§5.5)
Fehler	keine Zeilen `FEHLER` und `WATCHDOG` (Hinweise `LANGSAM` werden nur gemeldet)
Lauf	nach `R`: Zeile `Lauf endet` vorhanden
Zustand	letzter Rechnerzustand **vor dem letzten Netzschalter-Klick** (`RECHNER`-Zeile „… endet“): BZ, AC, AR, FE, FUE, BS, US, E, A gleich dem Soll
Speicher	die Zellen aus den `Prüfungen:` stehen nach dem Lauf in `trommel.txt`
Konsole	`grafik_stderr.txt` enthält weder `Traceback` noch `FEHLER`

Bei Zeitgrenze stehen die letzten 12 Zeilen des Debug-Logs im Bericht. `V1` und `V2` werden **nicht** geprüft (sie stehen nicht im Debug-Log). Beispiel einer Auswertungszeile: `OK  Tokens an den Rechner: soll G M+ E+ E- M- UE R | ist G M+ E+ E- M- UE R`.

5.7 Neuen Fall anlegen

Ein Fall ist ein Eintrag in der Liste `FAELLE` im Skript:

```
dict(name='pk-009_beispiel', text='kurze Beschreibung',
     zellen={0o0001: '00000010020'},                  # Anfangsbelegung, sonst 0
     folge='H G M+ E M- ST R',                        # wie in einer TEST-Testdatei
     soll='BZ=0003 AC=00000002000 AR=0020 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0',
     speicher={0o0001: '00000002000'})                # erwartete Zellen nach dem Lauf
```

Regeln:

1. **Zuerst im TEST-Interpreter bestehen.** Testdatei und Soll müssen dort Rückgabewert 0 liefern (das Skript prüft das selbst; `--nur-test` genügt zum Ausprobieren).
2. Der Name beginnt mit `pk-<laufende Nummer>`; er wird zum Ordnernamen.
3. Zellen sind Oktalzahlen (`0o…` als Adresse) und Wörter mit **11 Oktalziffern** (als Zeichenkette).
4. Nur Tasten aus §5.5 sind möglich. Geht eine Tastenfolge im Pult nicht (wie „ohne H“), wird sie anders gebaut und mit `ohne_test='Begründung'` gekennzeichnet (`pk-007`).
5. Nach jeder Änderung am Skript zuerst `--ohne-tk` (Trockenlauf), dann mit Fenster.

5.8 Grenzen

- **Nicht geprüft:** Aussehen, Klickflächen, Treffer- und Fokusprüfung des Fensters (die Wiedergabe umgeht das Tk-Mausereignis), Ton (`--STUMM`), das HW-Check-Fenster, `V1`/`V2`. Diese Punkte bleiben Handabnahme (§7).
- **Noch nicht in Klicks umgesetzt:** Selektoren `SP`, `BS`, `LS`, `SW_umschalt`, die Wippe `T`/`SW` (K0-Quelle) und die **Eingabetasten**. Das Skript lehnt solche Tasten ausdrücklich ab. Die Erweiterung wäre der nächste Schritt; Eingabetasten erfordern Programme mit Eingabe-Halt und Wartezeiten auf K0-Zeichen. (Die Eingabe über das SW-Fenster deckt §6 ab.)
- Windows ist nicht getestet (§9).
- Die Zeitabstände sind feste Pausen, keine Synchronisation auf das Log; auf langsamen Rechnern `--langsam` verwenden.
- Der Rechnerzustand wird **vor** dem letzten Netzschalter-Klick gelesen, weil der Auslauf den Rechner abbricht.

5.9 Ergebnisse

Datum	Umgebung	Ergebnis
18.10.2026	Entwicklungsumgebung, **ohne** Tkinter, Trockenlauf `--ohne-tk`	8 von 8 Fällen bestanden; die Auswertung wurde durch absichtlich falsches Soll, falsches Token und falschen Speicherwert geprüft (schlägt an)
18.10.2026	Linux, Python 3.11.4, **echtes Tkinter-Fenster** (Projektinhaber), Lauf `20261018_040829`	**8 von 8 Fällen bestanden** (163 Einzelprüfungen OK); keine `FEHLER`-, `WATCHDOG`- oder `LANGSAM`-Zeile in den Logs; Laufzeit je Fall 4,5 bis 8,2 s
18.10.2026	derselbe Rechner, erster Lauf `20261018_040638`	nach 3 bestandenen Fällen vom Anwender mit Strg+C beendet, vor der Zeitgrenze (48 s); kein Befund. Seither endet das Skript auch nach Strg+C mit Bericht, Summe und ZIP

5.10 Verweise

Thema	Dokument
Testformat des TEST-Interpreters (Trommeldatei, Tastenfolge, `#TROMMEL=LEER`, `Prüfungen:`)	`C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitte 2g, 3, 3a
Tasten, Wippen, Halter, Rollensperre des Pultes	`C8205-Interpreter-Bedienpult_mit_Schaltern.md`; Soll in der Beschreibung §3
Klickspur, Debug-Log, Wiedergabe	Kopf von `grafik_debuglog.py`; Schalter `--DEBUGLOG`, `--AUFZEICHNUNG`, `--WIEDERGABE` von `c8205_grafik.py` (`-h`)
Verzeichnisse und Skripte	`C8205-Projektstruktur.md`

(Die alte Verweistabelle nannte außerdem Dok. 01 §1.7, §1.8, §3.5, Dok. 06 und Dok. 08/`CHECKPOINT_Stufe2.md`; diese Anforderungsdokumente fallen mit Stufe 6 weg, ihr Inhalt steht in der Beschreibung.)

6 R300-Eingabe

Das Eingabeprogramm `r300_eingabe` („alle Zeichen des R300-Codes“, Version RE-001) prüft den Weg **Taste im SW-Fenster → K0-Warteschlange → Rechner → Schreibwerk** mit dem ganzen R300-Zeichensatz. Es ist die Prüfung zu den Anforderungen an das SW-Fenster (Beschreibung §5).

C 8205 – GRAFIK-Interpreter: Prüfverfahren

Quelle: C8205-Interpreter-GRAFIK_Pruefverfahren.md · im Wortlaut der Quelle

6.1 Bestandteile

Datei	Aufgabe
`r300_eingabe_erzeugen.py` (RE-001)	erzeugt aus **einer** Quelle (`c8205_interp.TABELLE1`) das Assemblerprogramm, `Tests/r300_eingabe.test`, die K0-Eingabedatei `Test-virtuelleGeraete/T-SW-Eingabe/r300_eingabe.txt`, die **Klickspur** und die **Sollfolge** (Zeichen, Umschaltzeichen, Code, Spurwort)
`r300_eingabe_trockenlauf.py`	Trockenlauf **ohne Tkinter**: Netz, Taste SW, G, R am Pult, getippte Zeichen durch die Logikschicht des SW-Fensters in den echten Rechner-Thread mit der Trommel `trommel_r300_eingabe.txt`
`r300_eingabe_grafik.sh` / `.bat`	GRAFIK mit `--HWCHECK` und `--WIEDERGABE`, rund 50 s; Faktor als Argument (`2` = alle Zeitabstände verdoppeln)
`r300_eingabe_lauf_und_auswertung.sh` / `.bat`	Lauf **und** Auswertung in einem Schritt; Rückgabewert 0 = ohne Befund, 1 = Befund, 2 = Lauf konnte nicht gestartet werden (z. B. kein Tkinter)
`r300_eingabe_log_auswerten.py` (RLA-002)	wertet das Debug-Log aus

6.2 Aufrufe

```
python3 -I testscripte/r300_eingabe_trockenlauf.py        Trockenlauf, Rückgabewert 0 = alles OK
sh testscripte/r300_eingabe_lauf_und_auswertung.sh [F]    echter Lauf mit Tkinter und Auswertung
sh testscripte/r300_eingabe_grafik.sh [F]                 nur der Lauf
python3 testscripte/r300_eingabe_log_auswerten.py [LOG [TROMMEL_NACH_LAUF [BERICHT]]]
```

Der echte Lauf legt die Trommel in eine **Kopie** (`r300_eingabe_trommel_nach_lauf.txt`); `Trommel/trommel.txt` bleibt unberührt. Ergebnisdateien: `r300_eingabe_debug.log`, `r300_eingabe_trommel_nach_lauf.txt`, `r300_eingabe_bericht.txt`. **Das Fenster während des Laufs nicht anfassen.**

6.3 Was die Auswertung prüft (je Tipp der Klickspur)

1. Kam im SW-Fenster die Taste an, die die Wiedergabe erzeugt hat? (Sonst: Wiedergabe-Artefakt, nicht Fehler des SW.)
2. Wurde ein vorgemerktes Zeichen, nämlich die zweite Eingabe nach einem Umschaltzeichen, dem Rechner wirklich geliefert? (Der Bericht nennt die gemessene Haltezeit und ordnet Werte unter 50 ms der Wiedergabe zu.)
3. Stimmt die Tokenfolge am Rechner mit der **Spur auf der Trommel** überein (ab Zelle 3000; Spurwort = Ausgabewort des Tokens)?
4. Stimmt die Anzeige im Fenster mit den angenommenen Zeichen und den Tokens überein (S3-04: keine Anzeige ohne Annahme, keine Annahme ohne Anzeige; das Programm gibt kein Echo aus)?
5. Wurden `ä ö ü ß §` (und `@ €`) verworfen, weil sie keinen R300-Code haben?
6. Lag der Fokus bei jedem Tipp im SW-Text?

Dazu die **blockierten Tasten** (kein `011` aktiv): Erwartung 0. Der Gravis steht als `[BEKANNT]` und zählt nicht (Wiedergabe-Artefakt, F-56). Rückgabewert 0 = nichts Auffälliges, 1 = mindestens ein Befund.

6.4 Erwartung und Ergebnisse

Die Sollfolge hat 93 Token; die Wiedergabe kann den Gravis nicht erzeugen, deshalb enden echte Läufe mit 92 Token und dem Gravis als `[BEKANNT]`.

Lauf	Umgebung	Ergebnis
Lauf 1 (Checkpoint 28, Linux, Python 3.11.4, Tk)	erster echter Lauf	Fokus 97/97; Tokens = Spur 92/92; Echo 86/86; ä ö ü ß § verworfen 5/5; Wiedergabe-Taste = angekommene Taste **91/97** (Befund B-3); vorgemerkte Zeichen **1/6** (Befund B-1)
F-54 (Checkpoint 40/41)	ohne Echo	Spur 91/91; Anzeige = Annahme = Tokens 85/85; Fokus 96/97 (Fenstermanager); Wiedergabe-Taste 91/97; zweite Eingabe **0/6** → Ursache: Wiedergabe ließ die Taste 0 bis 1 ms gehalten (Artefakt); Folge: Haltezeit 120 ms und AltGr in der Wiedergabe
F-55 (18.10.2026)	Wiederholung nach der Änderung	Spur 91/91; Anzeige 89/89; Fokus 97/97; zweite Eingabe 2/2; verworfen 7/7; Wiedergabe-Taste 96/97 (Gravis)

Lauf	Umgebung	Ergebnis
F-56 (18.10.2026)	Handlauf Gravis	Gravis ist tippbar und kommt am SW-Fenster als `grave` an; kein Fehler, keine Änderung nötig
F-58 (18.10.2026, nach G3-008)	echtes Fenster	Fokus 97/97, Wiedergabe-Taste 96/96 (Gravis `[BEKANNT]`), zweite Eingabe 3/3, blockierte 0, Spur 92/92, Anzeige = Annahme = Tokens (89); **keine Befunde**
Trockenlauf (19.10.2026, Entwicklungsumgebung ohne Tkinter)	Linux, Python 3.12.3	**10 OK, 0 FEHLER** (Token = Sollfolge 93, Verworfene, Ausgabe = nur die Meldung, Anzeige, Spur 93 Worte, hinter der Spur kein Wort)

6.5 Befund zu den Dateien der Klickspur und der Sollfolge

`r300_eingabe_grafik.sh` und `.bat` lesen die Klickspur aus `Tests/r300_eingabe_klickspur.txt`, `r300_eingabe_log_auswerten.py` die Sollfolge aus `Tests/r300_eingabe_soll.txt`. Beide Dateien hat der Projektinhaber am 19.10.2026 aus `Tests/` gelöscht (Dok. 11, „Beim Abnehmen festgestellt“), weil die Regressionsschleife der TEST-Seite sie als Tests mitzählte (Rückkehrcode 16). **Im Archiv liegen sie nicht mehr** (`Tests/` hat 199 Dateien, `r300_eingabe.test` ist die einzige mit `r300_eingabe`). Folge: Der **echte Lauf und die Auswertung können mit dem Stand des Archivs nicht starten**; der Trockenlauf läuft (er baut die Sollfolge selbst). Der Erzeuger schreibt die beiden Dateien wieder nach `Tests/`, womit der Fehler der Regressionsschleife zurückkäme. → §10 (B-4).

Entschieden am 19.10.2026 (Projektinhaber: „lassen“): Die Dateien bleiben, wie sie sind. Wer den echten Lauf oder die Auswertung braucht, erzeugt Klickspur und Sollfolge vorher mit `r300_eingabe_erzeugen.py` (sie landen in `Tests/`) und löscht sie danach wieder, damit die Regressionsschleife der TEST-Seite sie nicht als Tests zählt. Die vier Testskripte werden nicht geändert. Der Trockenlauf braucht die Dateien nicht.

7 Nur von Hand prüfbar

Alles, was Aussehen, Gehör, Zeitgefühl und das Verhalten des Fenstermanagers betrifft. Jede Zeile hat ein Sollverhalten; das Ergebnis meldet, wer prüft, mit Datum und Umgebung.

C 8205 – GRAFIK-Interpreter: Prüfverfahren

Quelle: C8205-Interpreter-GRAFIK_Pruefverfahren.md · im Wortlaut der Quelle

7.1 Übersicht

Nr.	Gegenstand	Sollverhalten (Quelle)	Verfahren	Stand
H-1	Aussehen und Klickflächen des Pults	Beschreibung §3	Sichtprüfung; Tasten einzeln klicken	vom Projektinhaber abgenommen (Stufe 2, 18.10.2026); **offen** OP-22 (Grafik „BS gedrückt“, fünf Foto-Abgleiche)
H-2	SW-Fenster: Tastenabbildung, Anzeige, Sperre	Handliste §7.2	`grafik_sw_tk_test.py --manuell --DEBUGLOG`	erledigt (A-3, 18.10.2026); `[ ] #`, Schnelltipp und Kopieren vom Projektinhaber gemeldet, nicht protokolliert
H-3	Fokus: SW-Text, Rückgabe an das Pult, HW-Check	Beschreibung §5.7, §8; hängt vom Fenstermanager ab	Tk-Test, WARNUNG beobachten	Tk-Test 1 WARNUNG (HW-Check, B-5) als bekannte Eigenschaft akzeptiert; „Fokus zum Pult zurück“ einmal nicht übernommen (12:27), später OK
H-4	Hörprüfung	Handliste §7.3	`./c8205_grafik.sh --HWCHECK` ohne `--STUMM`	Linux: Hörprüfung 1 (Befunde), Hörprüfung 2 **ok** (19.10.2026); Windows: Grundgeräusch, LL und LS gehört (26.10.2026), Rest offen
H-5	Taktgefühl: Reaktion auf Klicks im Dauerlauf R	Wirkung eines Klicks spätestens 50 ms (OP-17); Tastenklang: Verzögerung höchstens 50 ms (Mischer, B-1), Vorlauf 20 ms + Puffer 20 ms	Dauerlauf starten, H und Tasten klicken; Logik misst die Reaktionszeit von H (Block 3)	vorläufig entschieden, **Erprobung im Dauerlauf offen** (OP-17)
H-6	Puffer im SW-Fenster am echten Fenster	8 Eingaben, Ausnahme für Paare, kein Verfall (bestätigt 19.10.2026, OP-07)	Schnelltipp `a A a A` und längere Folgen; angezeigt = beim Rechner angekommen; mit `--DEBUGLOG` zeigt `SW Taste … voll` das Überlaufen	**nicht erprobt**

C 8205 – GRAFIK-Interpreter: Prüfverfahren

Quelle: C8205-Interpreter-GRAFIK_Pruefverfahren.md · im Wortlaut der Quelle

Nr.	Gegenstand	Sollverhalten (Quelle)	Verfahren	Stand
H-7	Eingabeklang beim Tastendruck	jedes Zeichen klingt **beim Tastendruck**, sobald es im Puffer ist (Beschreibung §6.2, OP-09, entschieden 21.10.2026); die Anzeige kommt erst bei der Annahme durch den Rechner; das **erste** Zeichen nach dem Einschalten (Paar mit Umschaltzeichen) klingt ebenfalls beim Drücken, angezeigt wird es nach dem Loslassen und der Annahme (§6.10 Nr. 3)	`testscripte/op09_eingabeklang.sh` starten (Test `sw_zwei_ziffern` auf frischer Trommelkopie, mit `--DEBUGLOG op09_debug.log`): Netz ein, G, R, als erstes Zeichen `2` tippen, danach `3`; Fenster schließen; Log lesen (`grep -E "Taste|Loslassen|Eingabeklang" op09_debug.log`): `SW Taste … -> folgt` und **sofort danach** `SW Eingabeklang …` (und `Stimmen im Mischer danach: n`, n größer 0); später `zweite Eingabe beim Loslassen -> geliefert` (Anzeige). Es muss **bei jedem Tastendruck** klicken, auch beim ersten und auch wenn der Rechner noch schreibt; ein Klick je Taste, keiner mehr bei der Anzeige. Steht die Zeile `Eingabeklang` da und es bleibt still: Tonstrom (Abspielprogramm)	Beobachtung 19.10.2026: kein Klick beim ersten Zeichen; Ursache 21.10.2026: Aufforderung des Testprogramms nach dem Umschaltzeichen (1,70 s); **Entscheidung (b) 21.10.2026, umgesetzt (G4-008/G4-010), Tests grün; Hören am Fenster steht aus**
H-8	`E+`: AR-Lampen zeigen BZ; G bei gehaltenem E	Beschreibung §3.8 Nr. 1, 2, 6	Pult: E niederdrücken (ohne M), Lampenreihe AR ablesen (= BZ), loslassen; mit gehaltenem M: AR-Lampen zeigen das erhöhte AR; G bei gehaltenem E: nur FÜ erlischt, BR/AR bleiben	OP-10 und OP-11 entschieden 21.10.2026 und umgesetzt (G4-005, G2-011); Tests `EGetrennt` in `grafik_pult_tests.py` (82 von 82); **Handprüfung am Fenster nicht durchgeführt**; Fortsetzung nach G im Überlaufhalt bestätigt (OP-41)
H-9	Ausfall des Tonstroms im Betrieb	Fehlermeldung, Abbruch mit Code 16 (S4-04)	Abspielprogramm im Lauf beenden	Start geprüft (G4-019); **Ausfall im Betrieb am echten Gerät offen**
H-10	Bildschirm: Fensterlage und -größe	1920 × 1080; Pult 1148 × 860 fest; Überlappen erlaubt (F-05)	Sichtprüfung; die gemerkte Fensterlage (in den Tests mit `C8205_FENSTERLAGE=aus` abgeschaltet)	Aussehen nicht beurteilt außer „keine Beanstandung gemeldet“; HiDPI, Mehrmonitorbetrieb, Wayland ungeprüft
H-11	Windows	→ §9		teilweise (26.10.2026), Rest offen
H-12	Lampenmittelung („ruhiges Bild“) im Dauerlauf	Fenster 40 ms; leuchtet ab 60 % „an“, dunkel bis 40 %, dazwischen bleibt der Zustand; bei Halt sofort genau (Beschreibung §3.5, OP-37)	Handliste §7.5	Logik durch Tests belegt (`grafik_pult_tests.py`, Klasse `Beruhigung`); Werte vom Projektinhaber **bestätigt 21.10.2026** (OP-37); **Handprüfung am laufenden Fenster nicht durchgeführt**

7.2 Handliste SW-Fenster (`grafik_sw_tk_test.py --manuell --DEBUGLOG`)

So gehen Sie vor:

1. `python3 testscripte/grafik_sw_tk_test.py --manuell --DEBUGLOG` starten (ab SWTK-003; Protokolldatei `sw_tk_test_debug.log`; die SW-Zeilen stehen auch im Terminal als `Protokoll: …`). Es öffnen sich drei Fenster: Pult-Attrappe, HW-Check, **SW-Fenster** (Titel „C 8205 SW-Fenster – Eingabe erwartet“). Das Terminal bleibt offen.
2. **Mit der Maus in das SW-Fenster klicken.** Nur dort tippen; was im Terminal getippt wird, zählt nicht.
3. Im SW-Fenster nacheinander tippen: `a`, `A`, `3`, Eingabetaste, Tab, Leertaste. Im Terminal erscheint nach jeder Taste eine Zeile „Rechner bekommt: …“ (Soll: Tabelle unten). Im SW-Fenster erscheint **jedes angenommene Zeichen**, auf der Schreibposition und in der Reihenfolge der Ausgabe des Rechners; **Umschaltzeichen erscheinen nie**. Getippt wird nur, solange das Fenster „Eingabe erwartet“ zeigt (sonst ist die Taste blockiert: keine Zeile „Rechner bekommt“, keine Anzeige). Einmal **langsam** (Umschalt gedrückt halten, dann das Zeichen) und einmal **schnell** tippen; der Abstand zwischen `Um. …` und dem Zeichen ist der des echten Tippens.
4. Weiter im SW-Fenster: `ä ö ü ß @ € §` (es darf weder eine Zeile „Rechner bekommt“ noch eine Anzeige erscheinen), dann `[ ] ~ #` und den Gravis; den Schnelltipp `a A a A` mit der Prüfung „angezeigt = beim Rechner angekommen“.
5. Zum **Beenden**: erst in das **Terminal** klicken, dann Enter. (Ein Enter im SW-Fenster beendet nichts, es ist die Taste WRZL.)
6. Zurückmelden: jede Taste, bei der die Zeile „Rechner bekommt“ vom Soll abweicht oder fehlt.

Erstes Zeichen (seit S3-12, G3-008): Das erste getippte Zeichen mit bestimmter Lage bekommt der Rechner **immer** mit Umschaltzeichen: `a` → `Um. KB`, dann `a`; bei `A` als erstem Zeichen `Um. GB`, dann `A`. ZR, TAB, WRZL gehen auch am Anfang allein. Danach gilt die Tabelle; der Vorzustand ist die Lage des zuvor getippten Zeichens („SW auf KB“ heißt: das zuvor getippte Zeichen hatte Lage KB, z. B. nach `a`, `3`, `%`, `/`, `+`, Gravis; „SW auf GB“: nach `A`, `!`, `(` usw.). Die Sollwerte gelten für eine **deutsche Tastatur**.

Taste (deutsche Belegung)	Zeichen	Soll, wenn SW auf KB steht	Soll, wenn SW auf GB steht
`a`, `3`, `%` (Umschalt+5), `/` (Umschalt+7), `+`	KB-Zeichen	nur das Zeichen	`Um. KB`, dann das Zeichen
`A`, `!` (Umschalt+1), `"` (Umschalt+2), `( )` = (Umschalt+8, 9, 0), <, > (Umschalt+<), `#`, `*` (Umschalt+Plus)	GB-Zeichen	`Um. GB`, dann das Zeichen	nur das Zeichen
~ (AltGr+Plus), `[` (AltGr+8), `]` (AltGr+9)	GB-Zeichen	`Um. GB`, dann das Zeichen	nur das Zeichen
Gravis (Umschalt+´, dann Leertaste; Tottaste; kommt als `grave` an, F-56)	KB-Zeichen	nur das Zeichen	`Um. KB`, dann das Zeichen
§ (Umschalt+3), $ (Umschalt+4), & (Umschalt+6), @ (AltGr+Q), € (AltGr+E), { } \ | (AltGr), ´, ä ö ü ß	kein R300-Zeichen	nichts, auch kein Umschaltzeichen	nichts
Eingabetaste, Leertaste, Tab		`WRZL`, `ZR`, `TAB` (ohne Umschaltzeichen)	dasselbe

Gravis-Handlauf (F-56, erledigt): Umschalt+´ und danach die Leertaste; ins Protokoll schauen, ob `Taste ignoriert … dead_grave` oder `Taste char='' … steht. Ergebnis 18.10.2026: Der Gravis kommt als grave an, es gibt keine Zeile dead_grave`; die Leertaste wird dabei vermutlich von der Tottaste verbraucht (nicht ausgeschlossen: nicht getippt; das Log allein entscheidet das nicht).

7.3 Handliste Hörprüfung

Aufruf: `./c8205_grafik.sh --HWCHECK` (ohne `--STUMM`), Lautstärke je Gruppe mit `--TON=pppp` (vier Ziffern 0 bis 9; Reihenfolge Pult, SW, LL, LS). Zu hören und zu bewerten:

Gegenstand	Sollwert (Quelle: Beschreibung §6, Konstanten §6.9)	Hörprüfung 1 (19.10.2026, nach Checkpoint 55)
`fade_in` beim Einschalten, Grundgeräusch, `fade_out`	`fade_in` 11,3 s, danach Grundgeräusch (Pegel 0,8), `fade_out` 2 s; Lampe N grün nach dem Aufblenden	Lautstärke gut
SW-Tastenanschlag	Pegel 2,0 (doppelt so laut wie zuerst)	„muss doppelt so laut werden“ → umgesetzt
Verzögerung des Tastenklangs	Vorlauf 20 ms, Puffer des Abspielprogramms 20 ms	„bemerkbar, vielleicht 0,1 s, nicht sehr viel“; nachzuprüfen
Mischung Grundgeräusch und Tastenklick	gut gemischt	gut
Leser	Schwelle 100 ms, 60 ms; Pegel LL 1,3 (OP-06, bestätigt)	nicht gehört bzw. nicht berichtet
Stanzer	50 ms, 5 s (Dauerstanzen), LS	nicht berichtet
angezeigte Eingabe, TAB, WRZL	Eingabe klingt wie Ausgabe; Umschaltzeichen stumm; TAB 400 ms	nicht berichtet
WR (Wagenrücklauf ohne Zeilenschaltung)	eigener kurzer Klang, 120 ms, Anfang des WRZL-Klangs ohne den lauten Schlag (OP-42); zu hören mit `python3 ton/sw_ton.py "ab<WR>cd\r"`: WR kurz, `\r` = WRZL lang	**neu 21.10.2026, nicht gehört**; ob der Schnitt (vor dem Schlag bei etwa 126 ms) der gemeinte „markante Ton“ ist, entscheidet das Ohr (Beschreibung §6.10 Nr. 6)
Schreibwerkzeit und Zeichenklang	82,5 ms (75 bis 90 ms; vorher 80 ms, 70 bis 90 ms), 100 ms	nachgestellt (S4-16)
Übergang `fade_in` zum Grundgeräusch	ohne Lücke	nicht berichtet

Hörprüfung 2 (Linux, 19.10.2026): ok (Meldung des Projektinhabers; Einzelbefunde zu den Zeilen oben nicht gemeldet). Das reicht für die Abnahme von Stufe 4 (B-6, B-3, B-4); eine zeilenweise Protokollierung gibt es nicht. Eine künftige Hörprüfung (auch unter Windows) hakt die Zeilen oben einzeln ab.

7.4 Trommel: Referenzwerte der Linux-Läufe (Dok. 02 §5)

Die Trommel wird in den Blöcken 15 (`test_c8205_trommel`, 11 Tests) und 16 (`make test-alle`, Trommel und Koinzidenz) automatisch geprüft. Dieser Abschnitt hält fest, **welche Werte als richtig gelten** und was nur von Hand oder nur unter Linux belegt ist. Die Zahlen stammen aus dem Lauf vom 16.10.2026 (gcc 13, ein Prozessorkern, `make test`: 8 Prüfgruppen bestanden). Ich habe sie **nicht neu gemessen**; Einzelwerte schwanken von Lauf zu Lauf leicht, maßgeblich sind die Sollwerte.

Prüfung	Sollwert	Ergebnis des Referenzlaufs
Lesen aller 32 Sektoren der Reihe nach	Werte stimmen, Wartezeit 0 bis 31	31 × 5 Ticks, 1 × 0
gleiche Adresse wiederholt (300 ×)	Wartezeit 0 (im Schlitz bedient)	Mittel 0,12 Ticks, 298 von 300 genau 0
logisch nächste Adresse, Sektor +1 (300 ×)	5 Ticks	5,03, 296 von 300 genau 5
physisch benachbart, +15 oktal (300 ×)	1 Tick	1,03, 299 von 300 genau 1
Zugriff auf den aktuellen Sektor (300 ×)	sofort bedient	300 von 300
Schreiben mit 14 Oktalziffern	gespeichert als 77777777777, Nachbarzeilen unversehrt	wie Soll
Bahnsperre (Hauptschalter EIN)	Bahn 0 gesperrt, Zeile unverändert, Wartezeit gemeldet; Bahn 1 schreibbar	wie Soll
Abbruch über Flag und `trommel_warten_abbrechen()`	`TROMMEL_ABGEBROCHEN`, Flag verbraucht	wie Soll
Adresse 010000, Zugriff nach Ausschalten	`TROMMEL_FEHLER`	wie Soll

Prüfung	Sollwert	Ergebnis des Referenzlaufs
alte 17-Byte-Datei	wird abgewiesen (erste Zeile auf stderr ist erwartet)	wie Soll
Takt	Tickzahl gleich der Uhr (2529)	2529; Aufwach-Verspätung im Mittel 25 µs, größte 1,9 ms

Zusätzliche Werkzeuge (Linux, einmalig gelaufen, nicht Teil des Sammellaufs): AddressSanitizer und UndefinedBehaviorSanitizer ohne Meldung; ThreadSanitizer nach der Behebung eines Datenwettlaufs (`req_wartet`) ohne Warnung. Die Beobachtungsvariable `trommel_aktuelle_sektornummer` ist absichtlich lockfrei lesbar (bestätigt 16.10.2026) und für ThreadSanitizer ein formaler Wettlauf; die Tests nutzen `trommel_aktueller_sektor()`.

Python-Seite (Block 15): Lesen mit Wartezeit 0 bis 31, Schreiben und Rücklesen mit Begrenzung auf 33 Bit, Bahnsperre, Fehler und `TrommelAbbruch`, Abbruch-Flag, Sektor und Statistik, Abweisung einer Datei im alten Format, Pfad mit Umlaut, Strg+C (Rückkehrcode 8, `ausschalten()` im `finally`, Trommeldatei danach mit 4096 gültigen Zeilen), Lader (nicht ladbar: Meldung und Code 16).

Nicht belegt: Die alten Testrahmen `testframe.c` und `testframe_trommel.c` kompilieren, wurden mit der neuen Fassung aber nicht ausgeführt. Der Strg+C-Test wird unter Windows übersprungen. Unter Windows besteht `test_c8205_trommel.py` (26.10.2026, §9); das übrige ist ungeprüft (W-7).

7.5 Handliste Lampenmittelung (H-12, OP-37)

Die Logik (Konstanten, Schwellen, Fenster) ist durch Tests belegt (`grafik_pult_tests.py`, Klasse `Beruhigung`: Konstanten 0,04 s, 60 %, 40 %; Schwelle 6 von 10 an, 4 von 10 aus; 5 von 10 behält den Zustand; neue Lampe im Zwischenbereich bleibt dunkel; Beurteilung erst nach 40 ms). Ob das Bild am Fenster **ruhig wirkt**, zeigt nur das laufende Fenster. Die Werte stehen als Konstanten `MITTEL_FENSTER_S`, `MITTEL_AN`, `MITTEL_AUS` in `grafik_pult.py` und sind am Fenster einzustellen (Beschreibung §3.5).

So gehen Sie vor (Vorschlag des Entwicklers, nicht durchgeführt):

1. Die Trommel sichern oder mit `--trommel EIGENE_KOPIE.txt` starten (OP-19); `./c8205_grafik.sh --trommel EIGENE_KOPIE.txt` (mit `--STUMM`, wenn der Hochlauf nicht abgewartet werden soll).
2. Netz einschalten, ein Programm laden, das **ohne Halt läuft** und die Lampen schnell wechseln lässt (zum Beispiel eine Zählschleife), und mit R in den Dauerlauf gehen.
3. **Beobachten:** Die Lampen der Adressen (AR), des Befehlsregisters und der Flags dürfen nicht flackern. Eine Lampe, die den größten Teil der Zeit „an“ ist, leuchtet dauernd; eine, die selten „an“ ist, bleibt dunkel; eine, die etwa gleich oft wechselt, behält ihren Zustand.
4. **H drücken:** Die Anzeige muss sofort den genauen Stand zeigen (kein Nachlaufen).
5. **Zurückmelden:** Flackert eine Lampe? Wirkt das Bild zu träge oder zu unruhig? Welche Lampe? Danach ist über Fenster und Schwellen zu entscheiden.

7.6 Trommelzeit der Befehlsschleife gegen die Dokumentation

Ein Programm, das nach der Magnettrommel-Dokumentation zeitoptimal liegt, findet seinen Sektor nur, wenn die Befehlsschleife zwischen zwei Zugriffen weniger Zeit braucht, als der Abstand vorgibt. Dokumentierte Mindestabstände (Wortzeit 104 µs): Sprung 4 Wortzeiten (416 µs), Einzelbefehl: Operand frühestens nach 2 Wortzeiten, Folgebefehl nach 5 (520 µs); Gruppen- und Wiederholungsbefehle: Phase IV je Wiederholung eine Wortzeit, bis 32. Ist die Schleife zu langsam, ist der Sektor vorbei und der Zugriff wartet bis zu eine Umdrehung (31 Wortzeiten, 3,2 ms) länger: Das Programm läuft richtig, aber langsamer.

Skript: `testscripte/grafik_sektor_pruefung.py` (SP-003). Aufruf aus dem Projektordner: `python3 testscripte/grafik_sektor_pruefung.py [--art sprung|einzel|gruppe] [--dauer S] [--d-von N] [--d-bis N] [--last N] [--mdverz ORDNER]`. Ergebnis: Tabelle und `testscripte/sektor_pruefung_ergebnis.csv`. Es ändert nichts im Projekt (Arbeitskopie der Trommel in einem Temp-Ordner). Je Zugriff wird die Wartezeit mitgeschrieben; ein *Fehlgriff* ist ein Zugriff, dessen Sektor vorbei war (Sprung: Wartezeit größer als der Abstand d; Einzelbefehl: ab 16 Wortzeiten; Gruppenbefehl: Abweichung vom festen Wartezeitmuster des 34-Zugriffe-Zyklus, weil dort lange Wartezeiten ohne Fehlgriff vorkommen). Zusätzlich wird die Rechenzeit in µs zwischen der Rückkehr eines Zugriffs und der nächsten Anforderung gemessen.

- `--art sprung`: Kette aus 32 Sprungbefehlen, Folgeadresse d Wortzeiten später (d = 1 bis 8).
- `--art einzel`: 31 Lade-/Speicherbefehle hintereinander, Operand d Wortzeiten nach dem Befehl (d = 1 bis 5); entspricht der Last von `grafik_zeit_messung.py` (Kennzahl „Warte/Tick“, ideal 2,5).

- `--art gruppe`: Gruppenbefehl 411 über die Spur 3000 mit Sprung zurück; zeigt, wie lange die 32 Wiederholungen von Phase IV dauern.

Ergebnis (Linux, Entwicklungsumgebung des Assistenten, Python 3.12.3, nur 1 CPU-Kern, nichts sonst laufend):

Messung	Ergebnis
Rechenzeit der Schleife	Sprung etwa 28 µs (99 % unter 50 µs), Einzelbefehl etwa 18 µs je Zugriff; also ein Zwölftel bis ein Sechzehntel des Abstands, Spitzen bis 1,5 ms
Sprung, d = 4 (Soll)	Fehlgriffe 0,01 % (3 s) bzw. 0,03 % (15 s); d = 5 bis 8: 0,00 % bis 0,06 %
Einzelbefehl, Operand 2 bis 5 Wortzeiten	Fehlgriffe 0,01 % bis 0,09 %, 2,54 bis 2,57 Wortzeiten je Zugriff in allen Fällen
Gruppenbefehl 411	Wartezeit der Wiederholungen im Mittel 1,12 Wortzeiten: Phase IV läuft mit der Trommel, eine Wortzeit je Wiederholung (wie dokumentiert). Der Zyklus hat 34 Zugriffe mit festem Muster (6, 27, 0, dann 31 mal 1 Wortzeit). Die 27 ist der Rücksprung 0001 nach 0000: Nach der Sektorfolge liegt 0000 27 Wortzeiten hinter 0001, das ist der Abstand der Trommel, kein Fehlgriff. Die frühere Zählung „etwa 3 %, rund einer je Zyklus“ (Schwelle ab 16 Wortzeiten, SP-002) war ein Messartefakt. Echte Fehlgriffe (Abweichung vom Muster): 0,8 % und 1,7 % in zwei Läufen zu 5 s. Die Reserve ist hier am kleinsten: Rechenzeit je Wiederholung im Mittel 19 bis 21 µs gegen 104 µs, 99 % unter 52 bis 63 µs; die Fehlgriffe passen zu den Spitzen (bis 2 ms) des Betriebssystems
Unter dem dokumentierten Mindestabstand	Sprung d = 1 bis 3: Treffer, Fehlgriffe 0,17 % bis 0,37 % (Dokumentation: 4). Einzelbefehl, Operand nach 1 Wortzeit: Treffer, Fehlgriffe 0,04 % (Dokumentation: 2). Das Modell bedient diese Zugriffe, das echte Gerät hätte den Sektor verpasst
Sprungkette mit 1 Rechenprozess als Last auf dem einen Kern, d = 4	Fehlgriffe 2,4 % (d = 2 bis 8: 1,8 % bis 4,7 %)

Lesart: Die Schleife ist schnell genug, solange der Rechner-Thread die CPU bekommt; die seltenen Fehlgriffe (weit unter 1 %) kommen von Spitzen von 0,2 bis 3 ms (Planer des Betriebssystems, Wechsel zwischen den Threads). Bei Last auf dem einzigen Kern steigt die Rate auf einige Prozent; das ist auf einen Rechner mit mehreren Kernen nicht übertragbar. Umgekehrt ist das Modell gutmütiger als das Original: Es braucht für einen Befehl nur etwa 0,2 bis 0,3 Wortzeiten Rechenzeit statt der dokumentierten 2 bis 5 und bedient deshalb auch Zugriffe, die das Original verpasst. Zeitoptimale Programme (Abstände nach Dokumentation) laufen mit der Originalzeit; Programme, die in diesem engen Bereich (Sprung 1 bis 3, Operand 1 Wortzeit nach dem Befehl) liegen, laufen im Modell um bis zu eine Umdrehung je solchem Zugriff schneller als am Original. Das betrifft auch die Zeit der Haltzeit-Zeile (8.3). Eine Erzwingung der dokumentierten Mindestabstände ist nicht eingebaut. **Entschieden 22.10.2026 (Projektinhaber): nicht erzwingen;** `trommel.c` **bleibt unverändert, es gibt kein neues C-Binary dafür.**

Grenzen: Gemessen wurde nur auf diesem einen Linux-Rechner mit einem Kern. Nicht gemessen: Windows, mehrere Kerne, Lauf mit Pult-Fenster und Bedienung (siehe `grafik_zeit_messung.py`), Abstände von Befehlen mit X-Abbruch, Eingabe- und Ausgabebefehle. Die Wartezeit des C-Moduls zählt in ganzen Ticks (104 µs); die Rechenzeit in µs stammt von der Uhr des Python-Prozesses und enthält das Aufwachen des Threads nach dem Zuschlag nicht. Auf der Zielmaschine sollte `python3 testscripte/grafik_sektor_pruefung.py` (und mit `--art einzel`) laufen; der Wert bei Sprung d = 4 bzw. Operand d = 2 ist das Ergebnis, das zählt.

7.7 Zeitmessung: Pult-Fenster, Lampenabtastung und HW-Check gegen die Befehlszeit

Frage: Stören Pult-Anzeige, Lampenabtastung und HW-Check-Fenster die Befehlsausführung in Echtzeit? (Das Original führt einen Befehl in 0,2 bis 0,4 ms aus, das sind 2 bis 4 Wortzeiten zu 104 µs.)

Aufbau (gelesen): Das Pult und das HW-Check-Fenster laufen im Tk-Hauptthread (Pult-Takt 40 ms; HW-Check ohne eigenen Timer, über `fenster.zusatz`). Der Rechner läuft im Thread `rechner`, die Lampenabtastung im Thread `lampenabtastung` (alle 2 ms), die Trommel in einem C-Thread (Sektorwechsel alle 104 µs, GIL beim Warten freigegeben). Python kennt keine Thread-Prioritäten. Jeder Trommelzugriff wartet in Echtzeit auf seinen Sektor; kommt eine Anforderung zu spät, kostet das bis zu eine Umdrehung (etwa 3,2 ms). Beeinflusst wird nur die Echtzeit, nie das Rechenergebnis. Der HW-Check liest `m.BZ`, `m.AC`, `m.AR`, `m.adr`, `m.OR` und den Trommelinhalt ohne Sperre (Absicht): im Dauerlauf können Zeilenwerte aus leicht verschiedenen Momenten stammen, im Halt ist der Stand exakt.

Skript: `testscripte/grafik_zeit_messung.py` (ZM-001). Aufbauten: `rechner` (Grundlinie), `abtastung`, `takt_python`, `tk_ohne_abt`, `tk`, `tk_hw`; jede Messung in einem eigenen Prozess. Aufrufe aus dem Projektordner: `python3`

`testscripte/grafik_zeit_messung.py [--dauer 10 --wiederholungen 5] [--varianten rechner,tk_hw] [--sw 0.0001] [--liste]`. Das Skript überschreibt `testscripte/zeit_messung_ergebnis.csv` bei jedem Aufruf (vorher sichern). Last: trommeloptimale Endlosschleife aus 32 Befehlen auf 0000 bis 0037, abwechselnd Laden (`211`) und Speichern (`240`), Operand 2 Wortzeiten nach dem Befehl, nächster Befehl 5 Wortzeiten danach; eine Verzögerung ab etwa 50 µs kostet 31 Wortzeiten, die Last ist also empfindlich. Kennzahlen: Zugriffe/s, mittlere Wartezeit je Zugriff in Wortzeiten (ideal 2,5), mittlere und größte Verspätung des Trommel-Threads in µs (die größte gilt seit Programmstart, nicht nur im Messfenster). Mess-Hilfen: `tr.T.statistik()` und `GrafikTrommel.wartezeit_summe`.

Ergebnis (gemessen, 20.10.2026):

Rechner	Ergebnis
Entwicklungsumgebung (1 Kern, Python 3.12, ohne Tk)	Abtastung −1,1 %, Pult-Takt −0,3 % Zugriffe/s gegen die Grundlinie
Rechner des Projektinhabers (i9-13900HX, Python 3.11.4, 5 × 10 s, ungepinnt)	Abtastung −0,5 %, Tk −0,7 %, Tk mit HW-Check −1,4 %; Verspätung überall etwa 5 µs; größte Verspätung höchstens 1,3 ms, nicht höher als in der Grundlinie
derselbe Rechner, ganzer Prozess auf einen Kern gepinnt (`taskset`; ungünstigster Fall)	Tk mit HW-Check −1,1 % bis −9,8 % Zugriffe/s; mittlere Verspätung höchstens 11,5 µs (Verlustgrenze etwa 50 µs); Wartezeit je Zugriff höchstens +5 %

Schluss: Die Lampenabtastung stört die Befehlsausführung nicht messbar; Pult-Takt und HW-Check kosten wenig; es wurde kein Einbruch gefunden. **Ein Umbau der Lampenabtastung ist nicht nötig** (bei Bedarf: Rohwerte in einen Ringpuffer und Auswertung im Tk-Takt, Schnappschuss an der Befehlsgrenze, C-Abtaster oder Anzeige in eigenem Prozess; nur die letzte Lösung beseitigt die GIL-Konkurrenz ganz). Auffällig, nicht erklärt: Mit einem Lastprozess auf dem Kern sind die gepinnten Werte besser als ohne (Vermutung: der Kern taktet sonst herunter). Im Einzelschritt und im E/A-Halt zeigt die Anzeige den genauen Stand direkt aus `m`.

Grenzen: Nicht gemessen sind Windows, ein Lauf mit Bedienung (Fenster ziehen, schnell wechselnde Lampen), die Rechenzeit-Reserve der Knoten direkt und andere Befehlsmischungen als die Lade-/Speicherschleife. Die Messung gilt für die genannten Rechner.

7.8 Einlesezeit des KEP und Trommelzeit

Das Einlesen des früheren `pj2-bis_minimalprogramm.kep` (7542 Zeilen; die Messung stammt von der alten Fassung, ersetzt durch `pj2-bis_minimalprogramm_v6.kep`) über das KEP dauert 54 s, über 90 % davon im Warten der Trommel. Die virtuelle Trommelzeit (`--trommelzeit alt|echt|schnell`, Standard `alt`; Beschreibung: GRAFIK_Beschreibung §7.8) zählt 521 241 Wortzeiten = 54,2 s, das ist dieselbe Zeit wie das C-Modul nach der Wanduhr. Der Interpreter hält also mit der Trommel Schritt; die Vermutung, das KEP verpasse wegen eines zu langsamen Interpreters seinen Sektor, ist nicht belegt. Die Schleife des KEP (Zellen 0020, 0030, 0031/0021) wartet je Durchlauf feste 22, 24 und 11 bzw. 6 Wortzeiten, zusammen etwa 70 Wortzeiten = 7,2 ms je Zeichen, fast genau ein Zeichen je 1/130 s (7,7 ms): Das KEP ist auf das Lesetempo des Lesers abgestimmt, nicht auf die kürzeste Zeit je Zeichen. Folgerung: Bei 130 Zeichen/s dauern 7542 Zeichen mindestens 58 s und die 12 139 Zeichen von `namen_raten.kep` mindestens 93 s, auch am echten Gerät. Gemessene 101 s im Fenster (statt 92 s vor dem Zeitmodell) sind nicht aufgeklärt; der Projektinhaber verfolgt das nicht weiter.

8 Abnahmekriterien der Stufen 1 bis 5

Kurzfassung der Kriterien und ihres Standes, als Referenz. Die Einzelheiten stehen in den Quellen (Dok. 07 bis 11), die mit Stufe 6 wegfallen.

8.1 Meilensteintabelle (S6-09, O6-6)

Stufe	Inhalt	Abnahme durch den Projektinhaber	Ausnahmen
1	Kern und Trommel (`c8205_grafik.py`, C-Modul), Geräte	**18.10.2026** („abgenommen“, nach Hinweis auf den Stand in Checkpoint 49)	Windows: Teilabnahme 26.10.2026 (§9); Standardannahmen (1) bis (3) unbestätigt
2	Pult-Fenster, Netzablauf, Rechnertöne, `--HWCHECK`	**18.10.2026** (ebenso)	Töne nicht gehört geprüft (dann Stufe 4); Windows; halber E-Befehl (OP-20); Dok. 06 nur zum Teil
3	SW-Fenster	**18.10.2026** („bis hierhin alles ok, abgenommen“, nach Checkpoint 46/48)	PDFs durch den Projektinhaber (A-6, später erledigt); Handliste `[ ] #`, Schnelltipp, Kopieren nur gemeldet
4	Töne (Mischer, Schreibwerk, Leser, Stanzer)	**19.10.2026** („ich nehme Stufe 4 ab“)	B-2 Windows: Bau erledigt, Dauerbetrieb offen; B-6 Windows teilweise (§9); Handbücher am 21.10.2026 auf Töne durchgesehen, keine Änderung nötig (OP-21, Beschreibung §10.5)
5	`--MODUS` TEST/GRAFIK, Standard GRAFIK	**19.10.2026** (ohne Windows)	A5-8 Windows: eigene, spätere Abnahme
6	Dokumentation konsolidieren	in Arbeit	–

Weitere belegte Meilensteine: erster echter Tk-Lauf `r300_eingabe` (Checkpoint 28); Pult-Tastentest PTK-001, 8 von 8 am echten Fenster (18.10.2026); Hörprüfung 1 und 2 (19.10.2026); Gesamtlauf 18 von 18 Blöcken am Rechner des Projektinhabers (19.10.2026).

Offen für S6-09: Der Einzelverlauf aus Dok. 00 §6 („Entwurf 1 bis 137“) ist hier **noch nicht** zu Anfangsterminen der Stufen verdichtet; die Tabelle nennt nur die belegten Abnahmen und Läufe aus den Dok. 07 bis 11.

8.2 Stufe 3 (Dok. 09 §4)

Nr.	Kriterium	Stand
A-1	Logiktests grün: `grafik_sw_tests`, `grafik_sw_ende_zu_ende`, `grafik_sw_pult_test`, VM-003	erfüllt (Checkpoint 47, wiederholt Checkpoint 49: 47/47, 15/15, 5/5, 4/4)
A-2	Lauf `r300_eingabe_grafik.sh` im echten Fenster, Auswertung ohne Befund	erfüllt (F-58, Checkpoint 47)
A-3	`grafik_sw_tk_test.py` mit echtem Tkinter ohne FEHLER; Handliste abgehakt	erfüllt (SWTK-004: 49 OK, 0 FEHLER; Handliste gemeldet)
A-4	`--HWCHECK` zusammen mit dem SW-Fenster am echten Bildschirm	erfüllt; Fokus (B-5) fenstermanagerabhängig, akzeptiert
A-5	Entscheidungen S3-12, S3-13, S3-14, S3-18 getroffen und umgesetzt	erfüllt (Checkpoint 44)
A-6	Dokumentation nachgeführt (0.03-016, PDFs, README)	Text erfüllt; PDFs durch den Projektinhaber
A-7	VM-002/VM-003 ohne `expectedFailure` grün	erfüllt (Checkpoint 37)
A-8	S3-04 neu umgesetzt: jedes angenommene Zeichen wird angezeigt	erfüllt (Checkpoint 47, F-58)

8.3 Stufe 4 (Dok. 10 §5)

Nr.	Kriterium	Stand	
B-1	Logiktests für Mischer, Leser, Stanzer, Schreibwerk ohne Audiogerät	erledigt (69 Prüfungen, gegen Python-Mischer und C-Modul)	
B-2	`tonmisch` baut unter Linux als statische Bibliothek, im Erweiterungsmodul; Windows (MinGW-w64)	**Linux erledigt; Windows: gebaut und eingeschaltet, Dauerbetrieb offen** (§9)	
B-3	Selektor SW, `--STUMM`, Strg+C/X/Schließen, Netz aus	echter Lauf mit Tk-Fenster: läuft (19.10.2026)	
B-4	Angezeigte Eingabe klingt wie die Ausgabe, auch bei Selektor SW	echter Lauf: läuft (19.10.2026)	
B-5	Fehlendes Abspielprogramm: Abbruch mit Code 16	Start erledigt (G4-019); **Ausfall im Betrieb offen** (H-9)	
B-6	Hörprüfung am echten Rechner, Konstanten nachgestellt	Linux: Hörprüfung 2 ok; **Windows teilweise** (26.10.2026: Grundgeräusch, LL, LS gehört)	
B-7	Bisherige Prüfungen bleiben grün, Tonstrom ändert kein Rechnerverhalten	erfüllt (19.10.2026, 18 von 18)	
B-8	**Puffer am echten Fenster, Klicks im Dauerlauf, Eingabeklang beim ersten Zeichen** nicht erprobt oder nicht entschieden; E+/BR/BZ und G bei gehaltenem E sind entschieden und umgesetzt (OP-10, OP-11), nur die Handprüfung H-8 fehlt	→ §7.1, H-5 bis H-8; OP-07, OP-17; OP-09: H-7, Beschreibung §10.5	siehe Beschreibung §10

8.4 Stufe 5 (Dok. 11 §4)

Nr.	Kriterium	Stand
A5-1	`grafik_alle_tests.sh` ohne `--schnell`, alle Blöcke grün	erfüllt, 19.10.2026: 18 von 18
A5-2	`python3 c8205_interp.py --MODUS=TEST Tests/0015-030.txt`: „Test wahr“, Code 0	erfüllt; Lauf durch den Entwickler in einer Projektkopie
A5-3	`python3 c8205_interp.py` ohne Schalter öffnet Pult und SW mit Ton	erfüllt (durch den Tk-Lauf mit Ton bei Stufe 4 gedeckt)
A5-4	`./c8205_interpreter.sh Tests/0015-030.txt` und `c8205_bis_test.sh` wie vor Stufe 5	erfüllt (Stand der Stufe 5 mit den damaligen Einzeltests `pj2-bis_*.txt`, inzwischen durch `Tests/pj2-bis_v6_*.test` ersetzt); BIS-Tests: 49 × Code 0, 1 × Code 6 (`pj2-bis_ps_zv_fehler.txt`, erwartet falsch)
A5-5	`grafik_gleichwertigkeit.py --alle` grün	erfüllt
A5-6	Alle Aufrufe von `c8205_interp.py` mit Testdatei tragen `--MODUS=TEST` (Suchbefehl)	erfüllt (G5-004)
A5-7	PDFs neu erzeugt	erfüllt (G5-009), bestätigt
A5-8	Windows (S5-12)	teilweise (§9)

Suchbefehl zu A5-6 (im Projektordner):

```
grep -rnE "c8205_interp(\.py)? " --include=*.md --include=*.py --include=*.sh --include=*.bat . | grep -vE
    "MODUS|--version|--help"
```

8.5 Stufen 1 und 2

Für Stufe 1 und 2 gibt es keine nummerierte Kriterientabelle; die Abnahme (18.10.2026) stützt sich auf die Läufe der Logiktests, den Gleichwertigkeitslauf (Stufe 1), den Stresstest und den Pult-Tastentest (§5) sowie die Handabnahme des Pult-Fensters (Stufe 2). Die bekannten Einschränkungen stehen in §8.1 und in der Beschreibung §10.4.

9 Windows (Teilabnahme 26.10.2026)

Windows ist **eigene Abnahme** (O5-3, O6-5). Sie läuft gegen dieses Verfahren, nicht gegen Dok. 10 und 11. Stand 26.10.2026 (Meldungen des Projektinhabers, nicht nachgeprüft): Das C-Modul ist unter Windows mit MinGW-w64 gebaut, geladen und die Haupttrommel getestet; `c8205_grafik.bat` startet, der Mischer (`winmm`) ist eingeschaltet. Offen bleibt der größere Teil (§9.2, §9.4). Bau, Fehlerbilder und Dateiablage: Beschreibung §9.2.

9.1 Was bisher gilt (Quellen)

Punkt	Stand	Quelle
Bau `tonmisch` mit **MinGW-w64** (statische Bibliothek, kein MSVC) und Einbindung in das Erweiterungsmodul	gebaut und im Modul eingebunden am 26.10.2026 (OP-01: Bau und Betrieb erledigt)	S4-02, B-2
`c8205_grafik.bat`: prüft `python` und `c-module\python\c8205_trommel*.pyd`; Bau: `c-module\python\bau_trommel_mingw.bat` (MinGW-w64); Rückgabewert 0, 8 (Strg+C), 16 (Start nicht möglich)	am 26.10.2026 gestartet (Start aus `neu\`); Rückgabewerte nicht gemeldet (OP-02)	S5-12
`c8205_interpreter.bat`	keine Meldung, nicht geprüft (OP-02)	S5-12
Ton: Mischer über `winmm` (Probe `winmm_pruefen()`), Startzeile nennt den Weg; Rückfall `winsound` mit Hinweis und Grund	Grundgeräusch, LL und LS gehört (26.10.2026); Schreibwerkton, Lücken, Lautstärke und `--TON` nicht geprüft (OP-03)	O4-18
Trommel: Plattformschicht `plattform.h` (Win32)	übersetzt, `test_c8205_trommel.py` besteht (26.10.2026); ob das aktive Warten dort entfernt ist, nicht geprüft (OP-04)	Dok. 02 §4.4
`c-module/build_msvc.bat` (baut `trommel.dll` und `testframe_koinzidenz.exe` mit `cl`)	im Kopf „NICHT getestet“; **nennt MSVC**, während S4-02 für `tonmisch` MinGW-w64 festlegt	–
Tk unter Windows und WSL, Bildschirmgrößen, Zwischenablage, X/Strg+C über die Pipe-Meldung	nie geprüft (OP-05)	Dok. 01 §8 Nr. 13, Dok. 04 Nr. 15

9.2 Abnahmekriterien Windows (Vorschlag, aus den Quellen abgeleitet)

*Diese Kriterien sind **Vorschläge des Entwicklers** (Kennungen W-1 bis W-9 neu in diesem Dokument, nicht im Register). Sie bündeln B-2, B-6, S4-15, S5-12 und A5-8 zu prüfbaren Schritten. Der Projektinhaber bestätigt sie oder ändert sie. **Stand 19.10.2026: nicht bestätigt; der Projektinhaber kannte sie nicht und hat nicht entschieden.** Sie sind für keine bisherige Abnahme nötig und werden erst gebraucht, wenn ein Windows-Tester da ist. Teile davon sind am 26.10.2026 beim Projektinhaber erledigt (Stand unten); bestätigt sind die Kriterien weiterhin nicht.*

Nr.	Kriterium	Aus	Prüfer
W-1	**Bau des Moduls:** `python setup.py build_ext --inplace` in `c-module\python` liefert `c8205_trommel*.pyd`; Bau von `tonmisch` mit MinGW-w64 gelingt und ist im Modul eingebunden	B-2, S4-02, S5-12	Tester
W-2	**C-Testrahmen:** Trommel, Koinzidenz und `tonmisch` laufen unter Windows grün (Wege: `make test-alle` mit MinGW-w64 oder `build_msvc.bat` für den Koinzidenz-Rahmen; welcher Weg gilt, ist zu bestimmen)	B-2	Tester
W-3	**Start GRAFIK:** `c8205_grafik.bat` öffnet Pult- und SW-Fenster; Rückgabewert 0 bei normalem Ende (Netz aus), 8 bei Strg+C, 16 bei fehlendem Modul oder Python	S5-12, A5-8	Tester
W-4	**Start TEST:** `c8205_interpreter.bat Tests\0015-030.txt` meldet „Test wahr“, Code 0 (wie A5-2, A5-4)	A5-4, A5-8	Tester
W-5	**Logiktests unter Windows:** die Skripte der Blöcke 1 bis 16 (§3.2) laufen mit `python -I` einzeln grün; wie der Sammellauf unter Windows gestartet wird, ist offen (`grafik_alle_tests.sh` ist ein Bash-Skript)	A5-1	Tester
W-6	**Ton:** Rechnertöne (`winsound`) hörbar; Hinweis beim Start, wenn der Mischer fehlt; mit gebautem Mischer Hörprüfung nach §7.3 (B-6)	O4-18, S4-15, B-6	Tester, Projektinhaber

Nr.	Kriterium	Aus	Prüfer
W-7	**Trommel:** der Lauf mit echter Wartezeit belastet den Rechner nicht durch aktives Warten (CPU-Last während eines Dauerlaufs beobachten; Blöcke 1, 3, 14 grün)	OP-04	Tester
W-8	**Tk-Verhalten:** Pult (1148 × 860) und SW-Fenster auf 1920 × 1080 mit Windows-Skalierung; Zwischenablage; Strg+C und X beenden sauber; `grafik_pult_tk_test.py` (ohne `--xvfb`) und `grafik_sw_tk_test.py` laufen (AltGr und Großbuchstaben setzt `zustand_x11()` dort **nicht**: erwartete Abweichungen sind zu klären)	OP-05, A-3	Tester
W-9	**R300-Eingabe:** `r300_eingabe_lauf_und_auswertung.bat` ohne Befund (setzt die Dateien aus §6.5 voraus: vorher erzeugen, danach löschen)	A-2	Tester

Windows-Zeilen in den Berichten tragen „Windows“ und die Python-Version; ein Linux-Ergebnis gilt nicht für Windows.

Stand der Kriterien am 26.10.2026 (nach den Meldungen des Projektinhabers, Python 3.13):

Nr.	Stand
W-1	Bau von `c8205_trommel.pyd` (mit `tonmisch`) und `c8205_ztmodul.pyd` gelingt; gebaut mit `bau_trommel_mingw.bat` und `bau_ztmodul_mingw.bat`, nicht mit `setup.py`
W-2	keine Meldung, nicht geprüft
W-3	`c8205_grafik.bat` startet; Rückgabewerte (0, 8, 16) nicht gemeldet
W-4	keine Meldung, nicht geprüft
W-5	keine Meldung, nicht geprüft (nur `test_c8205_trommel.py`: 23 Tests, 1 übersprungen)
W-6	Startzeile belegt; Grundgeräusch, LL und LS über den Mischer gehört; Hörprüfung nach §7.3 nicht durchgeführt; Schreibwerkton nicht geprüft
W-7	nicht geprüft (CPU-Last, Blöcke 1, 3, 14)
W-8	nicht gemeldet, nicht geprüft
W-9	keine Meldung, nicht geprüft

9.3 Ergebnis der Teilabnahme (26.10.2026, Rechner des Projektinhabers)

Baustein	Bau	Laden	Test
`c8205_trommel.pyd` (Trommel, Bahnsperre, Unterbrechung, Tonmischer)	gebaut mit MinGW-w64	`import c8205_trommel` ok	`test_c8205_trommel.py`: 23 Tests, OK, 1 übersprungen
`c8205_ztmodul.pyd` (Zusatztrommeln, C8205Z)	gebaut mit MinGW-w64	`import c8205_ztmodul` ok	kein Python-Test vorhanden
GRAFIK-Interpreter (`c8205_grafik.bat`, Start aus `neu\`)	–	startet; Startzeile „Hinweis: Toene unter Windows ueber den Mischer (winmm).“	Töne vom Inhaber **gehört**: Grundgeräusch, Lochbandleser (LL) und Stanzer (LS) über den C-Mischer mit `winmm`

Umgebung: Python 3.13, 64 Bit (`...\Python313`), `gcc` aus MSYS2 `mingw64` (`gcc -dumpmachine` meldet `x86_64-w64-mingw32`; die gcc-Version wurde nicht festgehalten).

Der übersprungene Test ist der SIGINT-Test (`test_c8205_trommel.py`, Zeile 81); er ist unter Windows absichtlich ausgenommen. Die Meldung `trommel_einschalten: Dateigroesse 69632, erwartet 73728 …` mitten im Lauf gehört zum Test, der eine Datei im alten 17-Byte-Format (4096 × 17 = 69632) anlegt und deren Abweisung prüft (`TrommelFehler`, Zeilen 70 bis 73). Sie erscheint auch unter Linux und ist kein Fehler.

Ton unter Windows (Beobachtung des Inhabers, 26.10.2026): Beim Start von `c8205_grafik.bat` erscheint die Zeile „Hinweis: Toene unter Windows noch nicht ueber den Mischer (winmm fehlt): nur Rechnertoene (winsound) und Schreibwerkzeit.“ Die Töne waren hörbar. Sie kommen aber aus dem Stufe-2-Zweig (`os.name == 'nt'` in `c8205_grafik.py`, Klasse `RechnerTon` in `grafik_ton.py`, `winsound`, ein Klang zur Zeit). Der C-Mischer `tonmisch` mit `winmm` wurde damit **nicht benutzt**, weil dieser Zweig ihn umging; Leser- und Stanzerton waren dort aus. Das Hören belegt also `winsound`, nicht den Mischer.

Änderung danach (26.10.2026): `c8205_grafik.py` prüft unter Windows zuerst mit `grafik_tonmisch_c.winmm_verfuegbar()`, ob das C-Modul mit `ton_*`-Methoden lädt und `winmm` sich öffnen lässt. Dann

läuft derselbe Aufbau wie unter Linux (Pult-, Schreibwerk-, Leser- und Stanzerton über den C-Mischer; Senke `CWinmmSenke`, `neuer_mischer_winmm()`), und die Hinweiszeile entfällt. Sonst gilt der alte `winsound`-Zweig mit der Hinweiszeile. Mit `--STUMM` wird `winmm` nicht geprüft. Fällt `winmm` im Betrieb aus, gilt wie unter Linux Code 16.
Ergebnis: Der Inhaber hat danach Grundgeräusch, Leser- und Stanzertöne gehört (Mischer mit `winmm` aktiv, Startzeile wie oben). Schreibwerkton, Lücken und Lautstärken (`--TON`) wurden nicht gemeldet und gelten als nicht geprüft.

Linux: Beide Module bauen mit der geänderten `plattform.h` unverändert; `test_c8205_trommel.py` läuft unter Linux mit 23 Tests ohne Auslassung und ohne Fehler (Python 3.12 in der Cloud-Sitzung, Build mit `gcc`, nicht mit `setup.py`). Die Änderungen stehen nur im `_WIN32`-Zweig und in `.bat`-Dateien.

9.4 Noch offen unter Windows (nicht geprüft)

- **Weitere DLLs:** Der Importtest lief in demselben Terminal, in dem der MSYS2-Ordner im `PATH` stand. Fehlende Laufzeit-DLLs aus `C:\msys64\mingw64\bin` wären dadurch nicht aufgefallen. Prüfen: `objdump -p c8205_trommel.pyd | findstr "DLL Name"` und Importtest in einem frischen Fenster ohne MSYS2 im `PATH`.
- **Python-Versionen:** Gebaut und getestet nur mit Python 3.13. Das Modul benutzt die Limited API (`Py_LIMITED_API=0x03080000`) und `python3.lib`; ob dieselbe `.pyd` unter 3.8 bis 3.12 lädt, ist nicht geprüft. Der Inhaber nutzt sonst Python 3.11.
- **Tonmischer unter Windows:** `tonmisch.c` mit `winmm` läuft (gehört: Grundgeräusch, LL, LS). **Nicht geprüft:** Schreibwerkton und Tastenklänge über den Mischer, Lücken und Aussetzer bei längerem Betrieb (Puffer 4 × 10 ms), Lautstärke und `--TON`, Verhalten ohne Audiogerät (Probe `winmm_pruefen()` meldet dann den Grund), Netz aus mit `fade_out`. Unter Linux belegt: die Änderungen stehen nur im Windows-Zweig; `grafik_stufe4_tests.py` besteht mit beiden Mischern (103 Tests), und der Sammellauf `grafik_alle_tests.sh --schnell` ist so wie vorher (siehe nächster Punkt). Ob `test_c8205_trommel.py` den Mischer aufruft, wurde nicht geprüft.
- **Vorhandener Fehlschlag unter Linux (nicht von dieser Änderung):** `testscripte/grafik_sw_ende_zu_ende.py` (Block „Stufe 3 Ende-zu-Ende“) meldet 2 Fehlschläge und 1 Fehler, mit und ohne die Änderung gleich (15 Tests). Ursache nicht untersucht.
- **Zeitverhalten:** Der Test hat 0,43 s gedauert und bestanden; Messwerte der Windows-Uhr (Aufwachverspätung, Zähler `aufwachen`, `nachgeholte_ticks`, Beschreibung §7.8) liegen nicht vor.
- **Zusatztrommel:** nur gebaut und importiert, kein Funktionstest. In `C8205Z/zt_modul` liegt als Test nur `testframe_zt.c` (C-Testrahmen).
- **GRAFIK-Interpreter und Pult unter Windows:** Der Start über `c8205_grafik.bat` läuft bis zur Tonhinweiszeile und die Töne waren hörbar (`winsound`). Ob das Pult-Fenster (Tk) vollständig bedienbar ist, wurde nicht gemeldet und nicht geprüft.
- `-static`**:** Die Skripte benutzen nur `-static-libgcc`; ob mehr nötig ist, hängt vom Ergebnis der DLL-Prüfung ab.

10 Bekannte Befunde

Nr.	Befund	Folge, Einordnung	Kennung
B-1	`sw_eingabe_leser_stanzer.test` lieferte im Gesamtlauf auf zurückgesetzter Trommel den Rückkehrcode 8 (Knotenlimit erreicht, Endlosschleife: Zelle 0000 sprang nach 0001 statt nach 02000)	**Erledigt 21.10.2026:** Test entfernt (Entscheidung des Projektinhabers); `k1_leser_stanzer` deckt Leser und Stanzer ab. Das Testhandbuch-PDF ist neu zu erzeugen, dann entfällt der Kasten „Achtung“	OP-18, G5-009, Dok. 11
B-2	**Trommeldatei wird von Testläufen verändert** (sie ist der Speicher des Rechners). Die Skripte dieses Dokuments arbeiten mit eigenen Kopien; im Probelauf vom 19.10.2026 (§10.1) blieb `Trommel/trommel.txt` unverändert. **Seit 21.10.2026 (OP-19, Variante b) prüft der Sammellauf das selbst** (Blöcke 19 und 20, §3.2); Läufe außerhalb des Sammellaufs im Modus TEST verändern die Datei weiterhin, das ist gewollt	für einen reproduzierbaren Lauf außerhalb des Sammellaufs vorher sichern oder `--trommel EIGENE_KOPIE`	OP-19
B-3	**ZIP-Archiv und Umlaut-Dateinamen:** Im ZIP heißen drei Dateien `…Eing#U00e4nge…`, `…#U00dcberblick…`, `…Durchf#U00fchrung…` statt `Eingänge`, `Überblick`, `Durchführung`. Wer das ZIP so entpackt, bekommt in den Blöcken 1 bis 4 `KonfigFehler: Knotendokumentation nicht gefunden: …Eingänge_der_Knoten.md`. **Nach dem Entpacken zurückbenennen** (`#U00e4` → ä, `#U00fc` → ü, `#U00dc` → Ü). Dasselbe Problem: Checkpoint 49 (A-1) musste die Dateien ebenfalls zurückbenennen; O6-1	kein Fehler des Projekts; Hinweis für jeden, der aus dem ZIP prüft	O6-1, A-1
B-4	**Klickspur und Sollfolge der R300-Eingabe fehlen im Archiv** (§6.5). Der echte Lauf und die Auswertung können so nicht starten; Neuerzeugung legt die Dateien in `Tests/` ab, wo die Regressionsschleife sie als Tests zählt (Code 16)	**Entschieden 19.10.2026 (Projektinhaber: „lassen“): so lassen**, Dateien bei Bedarf erzeugen und danach löschen (§6.5); die Skripte bleiben unverändert. Das Gerüst dieses Dokuments nannte „liegen in `Tests/` (G5-009)“; das ist überholt	G5-009, Dok. 11
B-5	**Gravis in der Wiedergabe nicht erzeugbar** (Tottaste); Läufe enden mit 92 statt 93 Token und dem Gravis als `[BEKANNT]`. Am echten Fenster tippbar (F-56)	Wiedergabe-Artefakt, kein Fehler; keine Änderung	F-56
B-6	**HW-Check-Fokus (B-5 aus Dok. 09):** Der Tk-Test meldet 1 WARNUNG, das SW-Fenster behalte den Fokus nicht; hängt vom Fenstermanager ab. „Fokus geht zum Pult zurück“ wurde einmal nicht übernommen (12:27)	als bekannte Eigenschaft akzeptiert; weiter beobachten	Dok. 09 §5d
B-7	**A5-2 und A5-4** liefen durch den Entwickler in einer Projektkopie, nicht am Rechner des Projektinhabers; vom Projektinhaber mit der Abnahme übernommen	zur Kenntnis	Dok. 11
B-8	**Puffer am echten Fenster, Klicks im Dauerlauf, Eingabeklang beim ersten Zeichen, E+/BR/BZ, G bei gehaltenem E** nicht erprobt oder nicht entschieden	→ §7.1, H-5 bis H-8; OP-07, OP-10, OP-11, OP-17; OP-09: H-7, Beschreibung §10.5	siehe Beschreibung §10

10.1 Probelauf vom 19.10.2026 (Entwicklungsumgebung)

Zur Probe des Verfahrens wurde der Sammellauf in einer **Kopie** des Archivs ausgeführt (Linux, Python 3.12.3, **ohne Tkinter, ohne Audiogerät**; die drei Dateien aus B-3 zurückbenannt):

Lauf	Ergebnis
`bash testscripte/grafik_alle_tests.sh --schnell`	**16 Blöcke bestanden, 0 fehlgeschlagen** (je Block nach der Reihenfolge in §3.2; die längsten: Stresstest 18 s, Tk-Trockenlauf 13 s, Ende-zu-Ende 11 s)
`python3 -I testscripte/r300_eingabe_trockenlauf.py`	**10 OK, 0 FEHLER**
Gleichwertigkeit `--alle`, `--neue-geraete`	**nicht gelaufen** (Block 17 und 18)
Tk-Tests am echten Fenster (§5, `grafik_sw_tk_test.py`), `r300_eingabe_lauf_und_auswertung.sh`, Hörprüfung	**nicht gelaufen**, hier nicht möglich (kein Tkinter, kein Ton)

Das Archiv selbst blieb unverändert; gearbeitet wurde in der Kopie. Dieses Ergebnis ersetzt keine Abnahme.

10.2 Kennungen

Kennung	Bedeutung
B-1 bis B-8 (in diesem Dokument)	Befunde dieses Prüfverfahrens; **nicht** zu verwechseln mit B-1 bis B-8 der Abnahme von Stufe 4 und B-1 bis B-6 der Befunde aus Dok. 09 (die stehen im Text mit „Dok. 09/10“ oder in Anhang B der Beschreibung)
H-1 bis H-11	Handprüfungen (§7.1)
W-1 bis W-9	Windows-Kriterien (§9.2), Vorschlag
PTK-001, SWTK-004, RE-001, RLA-002, VM-003	Versionen von Prüfskripten
OP-01 bis OP-24	offene Punkte der Beschreibung §10